Published: 06 May 2025
Online AM: 14 April 2025
Accepted: 01 April 2025
Revised: 01 March 2025
Received: 03 April 2024

SURVEY

Sandarśana: A Survey on Sanskrit Computational Linguistics and Digital Infrastructure for Sanskrit

ANAGHA PRADEEP, International Institute of Information Technology, Hyderabad, Hyderabad, TG, India

RADHIKA MAMIDI, International Institute of Information Technology, Hyderabad, Hyderabad, TG, India

Sandarśana: A Survey on Sanskrit Computational Linguistics and Digital Infrastructure for Sanskrit

ANAGHA PRADEEP, Language Technology Research Center, International Institute of Information Technology Hyderabad, Gachibowli, India
RADHIKA MAMIDI, Language Technology Research Center, International Institute of Information Technology Hyderabad, Gachibowli, India

Computational Linguistics is an interdisciplinary field of computer science and linguistics that focuses on designing computational models and algorithms for processing, analyzing, and generating human language. Over recent years, this field has made substantial progress. While its primary emphasis tends to center around widely spoken languages, there is equal importance in investigating languages that are not commonly spoken but have contributed immensely to the literature, culture, and philosophy of the society. Thus, this survey article comprehensively delves into the exploration of computational tasks undertaken for Sanskrit, an ancient language of the Indian sub-continent steeped in a wealth of literary heritage. The purpose of this study is to provide an overview of the progress made thus far in the computational analysis of Sanskrit, while also reviewing the current digital infrastructure that supports these efforts. Additionally, our study also identifies potential avenues for future research, serving as a reference for anyone interested in advancing their exploration in this field.

CCS Concepts: • **Computing methodologies → Natural language processing;**

Additional Key Words and Phrases: Sanskrit computational linguistics, Pāṇini, Aṣṭādhyāyī

ACM Reference Format:
Anagha Pradeep and Radhika Mamidi. 2025. Sandarśana: A Survey on Sanskrit Computational Linguistics and Digital Infrastructure for Sanskrit. *ACM Comput. Surv.* 57, 10, Article 254 (May 2025), 38 pages. https://doi.org/10.1145/3729530

1 Introduction

Linguists around the world have always been eager to analyse and frame theories on language learning. While Western scholars formulated the "universal grammar" theory [Hoque 2021], the Indian linguists were fascinated by the intricacies of how information is encoded in language, ensuring alignment between the speaker's intentions and the listener's understanding. Despite their differing approaches, both schools of thought underscored the significance of grammar in the process of language learning [Bharati et al. 2002].

Authors' Contact Information: Anagha Pradeep, Language Technology Research Center, International Institute of Information Technology Hyderabad, Gachibowli, Telangana, India; e-mail: anagha.pradeep@research.iiit.ac.in; Radhika Mamidi, Language Technology Research Center, International Institute of Information Technology Hyderabad, Gachibowli, Telangana, India; e-mail: radhika.mamidi@iiit.ac.in.

ACM 0360-0300/2025/05-ART254
https://doi.org/10.1145/3729530

Sanskrit, an ancient classical language of India within the Indo-Aryan language family, is the repository of ancient Indian culture, heritage, practices, and diverse knowledge systems such as astronomy, economics, mathematics, medicine, literature, and philosophy. Profoundly intertwined with the intellectual and cultural legacy of India, Sanskrit proficiency is essential for individuals interested in exploring this rich heritage. Therefore, the creation of computational tools for Sanskrit becomes imperative, serving as a valuable resource for enthusiasts aiming at deciphering texts related to the **Indian Knowledge System** (**IKS**). Not only that, Sanskrit exhibits linguistic evolution from its Vedic origins to the form meticulously described by the renowned grammarian Pāṇini in his masterpiece, the Aṣṭādhyāyī. Sanskrit is particularly distinctive among world languages due to the fact that Pāṇini's grammar (formulated around the 5th century BCE), was remarkably thorough at a time when the language had not yet been written (to the best of our knowledge). It systematically codified the phonemic system, morphological constructions, and syntactico-semantic structures in one comprehensive framework, establishing Classical Sanskrit as the standardized register for communication. Not only was this grammar highly descriptive, but it also became de facto prescriptive to such an extent that, from Vālmīki's Rāmāyaṇa onward, all classical Sanskrit literary works largely adhere to the rules outlined in this grammatical framework, resulting in nearly 25 centuries of homogeneous literature. Therefore, while the language itself became specifically suited for linguistic analysis, the Pāṇinian system gave a framework to develop original computational linguistic tools for such an analysis. Moreover, Pāṇini's work not only lays down a structural framework for Classical Sanskrit (samskṛta), it also encompasses insights into Vedic usages, demonstrating the differences between them [Huet 2003b].

Along with Aṣṭādhyāyī, the grammatical tradition of Sanskrit is also enriched by Kātyāyana's Vārttikas, and Patañjali's Mahābhāṣya. Additionally, we also have the three schools of philosophy viz Nyāya, Vyākaraṇa, and Mīmāmsā that contribute valuable theories of verbal cognition (Śābdabodha) supporting the semantics of Sanskrit at a deeper level. These linguistic theories and frameworks have significant relevance to the field of computational linguistics [Bharati and Kulkarni 2007].

Thus, leveraging Sanskrit's rich resources and its robust grammatical framework, numerous Sanskrit scholars, computational linguists, and computer scientists have worked diligently to create computational tools for analyzing Sanskrit (and other Indian languages). This collective effort has given rise to a new subfield called **Sanskrit Computational Linguistics** (**SCLs**), with Indian Cultural Heritage being its main application area (An overview of these tasks has been given in Figure 1).

This article seeks to outline a comprehensive overview of the wide range of computational tasks performed for Sanskrit. The exploration spans from rule-based methodologies that draw inspiration from the grammatical tradition to various Machine Learning experiments employing Sanskrit corpora. It also discusses the spectrum of digital infrastructure that Sanskrit has and subsequently identifies areas in which further research and refinement can be pursued. Although this survey article attempts to cover a wide range of tasks concerning Sanskrit processing by computer, it does not cover speech understanding and written character recognition.

2 Sanskrit Computational Linguistics: A Brief Background

Commencing in the 1980s, efforts were made to digitize Sanskrit texts. One notable achievement during this period was the creation of the first large collection of eighty texts by TITUS.[1] Subsequently, initiatives like The **Göttingen Register of Electronic Texts in Indian Languages** (**GRETIL**)[2] and the development of digital dictionaries emerged, driving efforts to digitize a larger

[1] http://titus.uni-frankfurt.de/indexe.htm
[2] https://gretil.sub.uni-goettingen.de/gretil.html

number of Sanskrit texts. Recognizing the importance of computational tools for Indian languages, the Indian Ministry of Communications and Information Technology actively advocated for their development starting with the **Technology Development for Indian Languages Program** (**TDIL**) [Goyal et al. 2012]. During the year 1999 - 2000 a significant initiative known as SanskNet was initiated by the Rashtriya Sanskrit Vidyapeetha in Tirupati, led by K. V. Ramakrishnamacharyulu. SanskNet aimed at unite traditional learning institutes, oriental research centers, manuscript repositories, and libraries to create a digital repository of Sanskrit texts accessible online to researchers [Kulkarni 2016]. Early efforts to mechanically process Sanskrit texts were very few and isolated. These included Pushpak Bhattacharyya's Sanskrit parser, developed using integer programming as part of his M.Tech thesis at IIT Kanpur. Then there was Lakshmitatachar's verbal cognition generator for Bhandarkar's Sanskrit primer which was developed at the Academy of Sanskrit Research, Melkote[3] in the early 1990s [Goyal et al. 2012]. In light of predominant linguistic theories favoring European languages and their structures, a novel approach to process Indian languages emerged with the "Akshar Bharati" group around 1995. This innovative approach adopted the Pāṇinian framework, providing a solution for Indian languages, and began with the digitization of approximately 475 books. Then there was an initiative by His Holiness Dr. Shivamurthy Swamiji of Sri Taralabalu Jagadguru Brihanmath, Sirigere (Karnataka) to simulate Pāṇini's Aṣṭādhyāyī through a software known as "Ganaka Ashtadhyayi".[4] Building on these early efforts, numerous initiatives have been launched to digitize and process Sanskrit texts, particularly given the significance of Pāṇinian grammar in shaping the linguistic understanding of this language [Goyal et al. 2012].

3 Challenges in Processing Sanskrit

Indian languages, including Sanskrit, exhibit a relatively flexible word order, where the position of words in a sentence does not convey information about their thematic role. Instead, semantic relationships between words are specified by declension suffixes [Bharati et al. 2002]. The Pāṇinian grammar is characterized by its syntactico-semantic nature. The adoption of such a grammatical framework would simplify the analysis of languages with free word order. Therefore, the kāraka[5] rules articulated by Pāṇini play a crucial role in the analysis of Sanskrit and have been utilized by systems, especially in the parsing algorithms. However, even before moving to the syntactic analysis of Sanskrit texts, there are challenges to be dealt with at the morphological level. This includes linguistic features such as

(1) Sandhi: The dominance of oral tradition necessitated the development of smoother and more fluid speech, leading to the emergence of the concept of "Sandhi" [Girrbach and Li 2022]. Sandhi refers to the phonological changes that occur either at morph or word boundaries [Natarajan and Charniak 2011]. This phenomenon is also common across other Indo-Aryan and Dravidian languages, as well as other languages in oral expression. Sandhi can be internal and external. Internal sandhi occurs at morph boundaries. For instance:
— Sanskrit: rāma + ina = rāmeṇa
While external sandhi occurs at word boundaries [Natarajan and Charniak 2011]. For example:
— Sanskrit: rāmaḥ + vigrahavān = rāmo vigrahavān
Sanskrit systematically encompasses both internal and external sandhi, and these phenomena can be found extensively throughout the language.

[3] https://vedavid.org/ASR/
[4] https://www.taralabalu.org/panini/
[5] Syntactico-semantic (or semantico-syntactic) relations between the verbals and other related constituents in a sentence [Bharati and Sangal 1993].

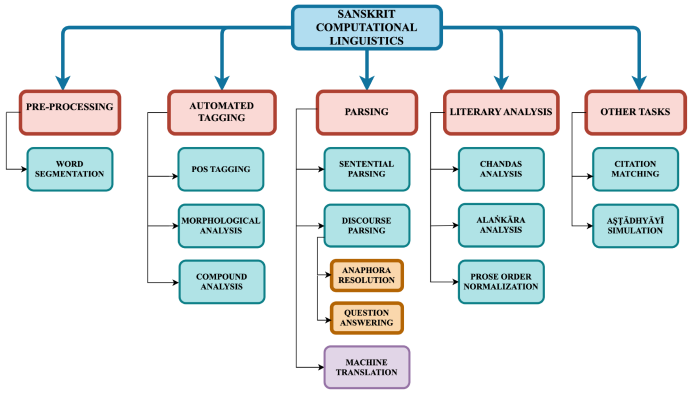

Fig. 1. Overview of tasks in Sanskrit computational linguistics.

(2) Samāsa: The merging of two or more words with distinct meanings into a single word expressing a unified meaning in a concise manner is known as samāsa or compounding. Compounding is a highly occurring phenomenon in Sanskrit. Compounds are similar to multi-word expressions and often do not have individual dictionary entries despite being treated as single units. The fact that they can be iterated as needed means that the language's words are not confined to a finite set, which makes morphology and syntax mutually recursive rather than separate layers for Sanskrit. Furthermore, during compound formations the phonological transformations (sandhi) between the compound components are a must. For example:
rāma + ālayaḥ = rāmālayaḥ (Rama's abode)
The word "rāmālayaḥ" is a compound (rāmasya ālayaḥ), for which the sandhi operation between the word boundaries (a and ā) is necessary.

The abundance of such morpho-phonemic and morphotactic features adds complexity to the analysis of Sanskrit. Nevertheless, considerable attempts have been made to resolve these challenges and to process Sanskrit texts.

Our article begins by exploring the most prominent pre-processing task for Sanskrit, which is word segmentation. This is followed by examining various taggers for Sanskrit including **part-of-speech** (**POS**) tagging, morphological tagging, and compound tagging. Subsequently, we delve into discussions about existing parsers. We discuss both sentence level parsers, and also applications of discourse parsing such as Anaphora Resolution and Question Answering. Followed by this is a section on Machine Translation. We then discuss tasks that are very specific to the analysis of Sanskrit literature such as Chandas Analysis, Alaṅkāra analysis and Prose Order Normalization. This is followed by sections on Citation Matching and Aṣṭādhyāyī Simulation. In the section on Aṣṭādhyāyī Simulation, we discuss how the grammatical treatise of Pāṇini has been made use of for simple generation tasks.[6] Additionally, the article has a section completely devoted to the digital infrastructure for Sanskrit such as computational platforms, digital libraries and repositories for training software.[7] In the final section, we examine potential avenues for further research and refinement in the field of SCL.

4 Word Segmentation

Sanskrit word segmentation involves compound segmentation along with incorporating mandatory sandhi splitting across words. However, resolving sandhi is non-deterministic in nature which

[6] A flowchart representing a brief overview of all the tasks has been provided in Figure 1.
[7] The datasets used for each task have been discussed in Section 12.2.

makes segmentation a complex task in Sanskrit. As the initial stride toward meaningful Sanskrit processing starts with segmentation, we have several segmenters spanning from rule-based to statistical and neural network-based. We will go through them.

Sanskrit Heritage Site (SH) - Huet [2003a] has developed a segmenter using the Zen toolkit [Huet 2005]. This toolkit is widely used in computational linguistics as it uses the data structure of annotated tries (lexical trees) to represent the lexicon in the form of **Finite State Transducers** (**FST**) [Goyal and Huet 2013]. The segmenter deals with both internal and external sandhis. The internal sandhi production is typically a part of the morphological engine where the inflexional forms are derived by processing the internal sandhi during the word formation. It consists of paradigm schemas where the morph parameters such as number or case are mapped to a set of possible suffixes. This is further mapped to the lexicon [Huet 2004]. On the other hand, the external segmenter uses rewrite rules (using the rules stated by Pāṇini) that are applied to all potential combinations of interactions involving phoneme strings with a length of one or two. Intra-compound segmentation is also handled under the external sandhi segmenter, with certain exceptions [Goyal and Huet 2013].

Automatic Sanskrit Segmentizer - Mittal [2010] propose two techniques for automatically segmenting Sanskrit words using 25,000 unsandhied word pairs and 2650 sandhi rules. The first method creates an FST incorporating sandhi rules, but its performance slows as the FST size increases. To address this, the study introduces an **Optimality Theory** (**OT**)-based approach, with components like GEN, CON, and EVAL to generate, constrain, and evaluate segmentation candidates. The article's findings indicate that both of these novel approaches outperform the baseline method, which assumes that each word can only be segmented into two constituents. However, it is the OT-based approach that stands out as it delivers satisfactory results in terms of both processing time and the quality of segmentation. Additionally, it is worth highlighting that the baseline system and the OT method both employed the extracted 2650 rules, on a test data of parallel corpus containing 2510 words. In contrast, the rule-based method utilized only 500 rules on the same test data.

University of Hyderabad (UoH) Model - In addition to the model discussed previously, Kumar and Kulkarni [2010] have used a similar approach for compound segmentation using OT. However, a key distinction in this model is that it takes into account the cumulative product of the probabilities associated with the occurrences of each word and the product of probabilities for the sandhi rules. In contrast, the previous model focused on the summation of probabilities for consecutive words (bigrams) and then multiplied this result by the probability of the corresponding sandhi rule.

Statistical Sandhi Splitting - Like Mittal [2010] and Kumar and Kulkarni [2010], Natarajan and Charniak [2011] also perform a statistical sandhi and compound splitting using OT. However, their approach differes from that of the other two. In their article, they explain how they modified the posterior probability function to achieve better results, reaching a precision of 70.52. The article also proposes a new algorithm for Sandhi splitting based on Bayesian word segmentation methods, which is further based on the Dirichlet process. They experiment with both unsupervised and supervised methods for the algorithms and conclude how the supervised surpasses all other systems.

RNN for Compound and Sandhi Segmentation - Hellwig [2015] introduces a novel approach for sandhi resolution as well as compound splitting using the **Recurrent Neural Networks** (**RNNs**). This approach relies solely on gold transcripts of correct string splits for training. The model does not make use of any extra external resources, such as language models or morphological or phonetic analyzers. As a result, it is particularly suited for languages that lack such resources. The aim of the algorithm as a whole is threefold; split compounds at the correct position, resolve sandhi, and produce the sandhi rules as per the definitions provided. The article

defines sandhi resolution and compound segmentation as a sequence labeling task for which they make use of an RNN that achieves an accuracy of 93.24. The data used for training the model was extracted from the corpus of SanskritTagger [Hellwig 2007].

Segmentation using Path Constrained Random Walks - In addition to the word co-occurrence feature-based segmentation method, Krishna et al. [2016a] have attempted to integrate segmentation using morphological features. This approach treats the segmentation challenge as a query expansion problem, where it iteratively chooses one segment from a set of competing segmentations (obtained from SH) and then proceeds to select the subsequent correct segment based on information from the expanded context. The algorithm concludes its process when there are no more candidates left for evaluation. To identify the correct segmentation, the authors employ the **Path Constrained Random Walks** (**PCRW**) framework alongside linguistically motivated **Inductive Logic Programming** (**ILP**) formulations. Through a series of experiments, the article demonstrates how this model has surpassed the previous ones in terms of performance with a precision score of 92.38. The dataset used to train the language models for this study includes a tagged corpus from DCS. It consisted of 4,30,000 segmented sentences with 66,000 unique lemmas.

Long Short-Term Memory (LSTM) based word segmenter - Reddy et al. [2018] propose a complete data-centric approach for the segmentation task. They use an encoder-decoder framework for the segmentation task and a deep Seq2seq model using multi-layered LSTMs (with attention and without attention) for the prediction task. The model is trained such that the conditional probability of predicting the unsandhied sequence given its corresponding sandhied sequence is maximized. The data used in this study includes 1,05,000 parallel strings from DCS, same as the one used by Krishna et al. [2016a]. The model is compared to two other baseline systems supervised PCRW by Krishna et al. [2016a] and GraphCRF. Of the four models, the article reports that the Seq2seq model with attention performs better with a precision score of 90.77.

IBM Model - Aralikatte et al. [2018] explore yet another deep learning model for compound segmentation. As discussed earlier, sandhi splitting is an essential part of compound segmentation. The article proposes an architecture called **Double Decoder RNN** (**DD-RNN**). The goal of this model is an automated split generation by learning the split locations of the compounds. The model is a deep bi-directional character RNN encoder with two decoders and attention. The first decoder is a location decoder that predicts the split location of the compounds, while the second one is a character decoder that generates the split words. The dataset used to train the model consists of 71,747 words and their splits. This includes the UoH corpus and the SandhiKosh corpus. The results of the model were compared with existing publicly available tools such as the UoH splitter [Kumar and Kulkarni 2010], the JNU splitter [Kumar 2007], and the Sanskrit Heritage splitter [Huet 2003a], and with other standard RNN architectures. The article reports the accuracy of the model to be 95% for location split and 79.5% for prediction split.

Word Segmentation Using CNN -Hellwig and Nehrdich [2018] have developed a deep-learning model designed to convert a sentence into segmented tokens (unsandhied tokens). Their approach combines character-level convolutional and recurrent components. While the recurrent components capture sentence-level information, the convolutional components replace traditional n-gram extraction methods. They propose three distinct models: **Convolution Recurrency** (**crNN**), **Recurrency Convolution** (**rcNN**), and rcNN with shortcuts. In the initial phase, these three models were evaluated for model selection. Subsequently, the selected model (rcNN with shortcuts) was compared against baseline models. These baseline models were also trained on a re-analysed dataset taken from the DCS created specifically for this study. In total, the dataset comprised of 5,61,596 sentences. The article concludes by stating that the rcNN shortcut model outperforms the other baseline models in terms of performance with an accuracy of 96.6%.

Energy Based Model - Krishna et al. [2018] have endeavored to tackle word segmentation and morphological tagging challenges in Sanskrit by employing an **Energy-Based Model** (**EBM**). The study utilized data sourced from the DCS. The EBM shares similarities with graph-based parsing techniques. In the process, the word splits obtained from the **Sanskrit Heritage** (**SH**)[8] reader serve as nodes in the graph. The SH offers a plethora of potential word splits, and to manage this abundance, a maximum clique approach is adopted for comprehensive segmentation. Consequently, the graph structure removes the inherent sequential aspect of inputs, while the maximum clique policy enables holistic consideration of the entire context. The study further comes up with 528 morphological constraints that are used for automatic tagging. Evaluation of the model was based on how well the inflectional forms of the segmented words were predicted (WPT) and also on an efficient joint task that comprised of word segmentation and morphological tag prediction (WP3T). In comparison to other baselines used in the study such as SupervisedPCRW, EdgeGraphCRF, Seq2seq, Lattice-EBM, and Tree-EBM, the Clique-EBM performs the best for both the tasks with a precision score of 96.18 for the former and 91.35 for the latter.

Ranking the SH solutions - The SH segmenter lists all possible segmentations that could sometimes lead to overgeneration. In situations where the output is excessively detailed, only a condensed summary of the split, phase, and transition is displayed. Krishnan and Kulkarni [2022] propose a probability function aimed at prioritizing solutions and highlighting the most valid ones at the top of the list. For evaluation, a test dataset comprising 21,127 short sandhi expressions is utilized to compare the performance of both the original and modified segmenters. The results indicate that the modified segmenter achieved a correct solution as the first choice 89% of the time, showcasing a notable improvement of 36% over the original segmenter. This was further improved by Krishnan and Kulkarni [2022] who introduce a ranking mechanism to prioritize the solutions and retain segments that are a part of the most probable segmentations. The study highlights that the major challenge faced by the rcNN model [Hellwig and Nehrdich 2018] and the introduced ranking mechanism lies in handling the compositionality of compounds. Additionally, the study also contributes a new annotated dataset consisting of 1,30,000 sentences sourced from the DCS.

Neural Compound-Word Sandhi Splitting - In order to address the low accuracy problem of the existing models, Dave et al. [2021] come up with a machine learning approach to solve external sandhi splitting. The problem is addressed in two steps. The first step involves predicting a sandhi window of the compound words using an RNN model. The second step involves splitting the predicted sandhi window into two different words using the seq2seq model. The model achieved an accuracy of 86.8% which was reported to be better than a few models discussed previously which includes the UoH model, SHR model, and DD-RNN model. The article also talks about Sandhi generation task and proposes two approaches for the same. As the results achieved from the initially proposed approach where the model expects two words as input were poor, the second approach was initiated. This involved addressing the issue by extracting the last n characters from the first input word and the first m characters from the second input word, facilitating Sandhi between the two resulting smaller words. Optimal results were obtained with n = 5 and m = 2 for the Seq2seq model. The neural network model was trained using data sourced from the UoH corpus. This dataset encompasses over 1,00,000 Sandhi examples out of which 81,029 were used after filtering. The results significantly highlight the commendable performance of Neural networks in the Sandhi generation task.

WSMP Hackathon Segmenter - As part of the WSMP Hackathon in 2022, a model was developed to tackle both word segmentation and morphological parsing [Girrbach and Li 2022]. The

[8]Has been discussed in Section 12.

study was structured around three primary tasks: (1). word segmentation, (2). morphological parsing, and (3). a joint task combining both. The word segmentation task was further divided into two subtasks–identifying appropriate word boundaries and retrieving unsandhied word forms. The approach for word segmentation treated it as a sequence labeling problem. Once the predicted sequence is obtained, the segmentation is extracted accordingly. This simultaneous determination of word boundaries and unsandhied word forms is integral to the word segmentation task. For the second task, the obtained word segmentations were input to the morphological parser to predict the stem and morphological tag for each word. In the final task, the objective was to obtain the unsegmented form of a sandhied sentence as a single string, along with its morphological analysis. This was achieved using a pipeline approach that was end-to-end trainable. The dataset provided by the hackathon organizers, was taken from the DCS, as indicated by them. The model achieved a precision score of 80.25 for the final task, the highest among all the models in the hackathon.

TransLIST - Sandhan et al. [2022b] propose a Transformer based Linguistically Informed Sanskrit Tokenizer. This is a character-level sequence labeling that includes latent word information from the SH as the first stage. As a part of the second stage, they use soft-masked attention to prioritize potential candidate words. The soft-masked attention of the Transformers enables an effective interaction between the characters and the latent word information. In the third stage, a path ranking algorithm is introduced in order to rectify candidate words that are not a part of the solution space using the SH. The study incorporates two subsets from the DCS corpus, with the model attaining a precision of 98.8% on the first dataset and 97.78% on the second.

ByT5-Sanskrit - Nehrdich et al. [2024] introduce a new unified pre-trained model, Byt5-Sanskrit designed for segmentation, lemmatization, dependency parsing and OCR post-correction tasks. For pre-training, the authors make use of Sanskrit data from the Sangraha dataset [Khan et al. 2024] which is augmented with data from GRETIL and Digital Sanskrit Buddhist Canon.[9] The model is further fine-tuned using annotated data from the DCS. The performance of the model for segmentation was assessed by fine-tuning it on established segmentation tasks (rcNN and TransLIST), where ByT5-Sanskrit outperforms the other two models with a perfect match score of 90.11, 93.83, and 94.29 on three different datasets. The lemmatizer achieves a perfect match score of 79.88 on the sentence level and 83.96 on the paragraph level on the DCS data. The model's dependency parsing was evaluated on the Vedic Sanskrit dependency parsing task, and was compared against the biaffine architecture previously used by Hellwig et al. [2023]. It showed higher performance with an **unlabelled attachment score** (**UAS**) of 89.04 and a **labelled attachment score** (**LAS**) of 84.58. Additionally, for the joint tasks of segmentation, lemmatization and dependency parsing, ByT5-Sanskrit achieves stronger performance at the paragraph level with a perfect match of 74.31%. This is reported to be a better score than at the sentence level.

In some cases, particularly with OCRed data, spell checking is necessary before performing word segmentation. There are a couple of spell checkers available for Sanskrit: The first, introduced by Tapaswi et al. [2012], is a rule-based spell checker that relies on rules derived from both morphology and orthography. The second, presented by Prasanna [2022], involves a Sanskrit spellchecking dictionary designed for Hunspell,[10] utilizing the word and paradigm model. This study examines Hunspell's compatibility with Sanskrit, detailing its dictionary format and spellchecking rules based on Paninian grammar. It also introduces a web interface for Sanskrit spellchecking and evaluates both the dictionary and interface.

[9] https://www.dsbcproject.org/

[10] Hunspell is a free open-source spellchecker and morphological analyser library. It is a command-line tool. (https://github.com/hunspell/hunspell/)

Table 1. Word Segmenters

Sl.No	Model	Year	Type	Data
1	SH	2003	Rule-based	-
2	Mittal	2010	Rule-based (a) Statistical (b)	TDIL Consortium Project
3	UoH	2010	Statistical	TDIL Consortium Project
4	S3	2011	Statistical	TDIL Consortium Project
5	Hellwig	2015	Neural	Sanskrit Tagger
6	PCRW	2016	Neural	DCS
7	seq2seq	2018	Neural	DCS
8	IBM	2018	Neural	UoH Corpus and Sandhikosh
9	CNN	2018	Neural	DCS (SEGMENTATION)
10	EBM	2018	Neural	DCS
11	Sriram	2020	Statistical	TDIL Consortium Project
12	Dave	2021	Neural	UoH Corpus
13	WSMP Hackathon	2022	Neural	DCS (HACKATHON)
14	TransLIST	2022	Neural	SIGHUM, DCS (HACKATHON)
15	Byt5-Sanskrit	2024	Pre-Trained	Sangraha and DCS

Word segmentation as a pre-processing task has been well investigated for Sanskrit, with each successive model striving to introduce innovations in their approaches to address challenges overlooked by previous ones.[11] The neural models have proven adept at handling out-of-vocabulary words, which rule-based or statistical systems struggled with. Before the release of Byt5-Sanskrit, rcNN and TransLIST were considered the **state-of-the-art** (**SOTA**) models for word segmentation. However, Byt5-Sanskrit, being an extension of rcNN, now holds the SOTA position. Both TransLIST and Byt5-Sanskrit are suitable for use on real-world data for segmentation. However, it is important to note that TransLIST is incompatible with the DCS dataset due to its complex pre-processing pipeline, even though it performs exceptionally well on the SIGHUM and Hackathon datasets. Byt5-Sanskrit, on the other hand, demonstrates consistent performance across various datasets, including DCS, highlighting that lexical pre-processing is not necessary to achieve competitive results.

5 Automated Tagging

5.1 Part-of-speech (POS) Tagging

The process of assigning a part-of-speech tag to each word in a given sentence is known as POS tagging and is an important pre-processing task in NLP [Kumawat and Jain 2015].

Rule-Based POS Tagging - A simple rule-based tagger for Sanskrit was introduced by Tapaswi and Jain [2012]. The model stores about twelve POS tags in its database and the tags are assigned through hand-crafted rules. In the first stage of the algorithm, lexical rules are utilized to assign tags, followed by the application of context-sensitive rules in the second stage for unknown words and incorrectly tagged words. A manual evaluation of 100 words at each stage yielded precisions of 90% and 91%, respectively.

Deep Learning POS Tagger with Character Level Embeddings - Premjith et al. [2018] conducted experiments using a deep-learning-based POS tagger for Sanskrit. The study framed the problem as a character-level sequence labeling task to minimize memory requirements. Various deep-learning models, including RNN, LSTM, GRU, and their bidirectional versions were employed

[11] See Table 1 for a brief summary.

with modified hyperparameters. The study utilized the Universal Dependencies tagset, comprising sentences from Pañcatantra and featuring 16 tags. Training involved around 34,270 words, while testing was performed on 4,231 words. While all the models exhibit strong performance with accuracy scores surpassing 95%, the Bi-directional GPU model achieved an accuracy score of 97.86% and outperformed the others.

Deep Learning based Unsupervised Tagging - Srivastava et al. [2018] introduce an innovative method for assigning POS tags to Sanskrit words. To address the challenge of absence of extensive annotated corpora for Sanskrit, the study advocates for an unsupervised approach, employing the untagged Sanskrit Corpus created by JNU. The research delves into various techniques to represent each Sanskrit word through a multidimensional vector, incorporating word embeddings, character n-grams, autoencoders for dimensionality reduction, and bidirectional LSTM autoencoders to capture semantic information and model sequential dependencies. These compressed encodings are subsequently clustered using K-Means clustering, leading to the assignment of POS tags based on the clusters. The model undergoes testing on a tagged dataset comprising 115,000 words prepared by JNU. The evaluation encompasses multiple vectorization methods and dimensionality reduction algorithms, ultimately concluding that the character n-grams vectorization method, coupled with Bi-LSTM autoencoder, yields the highest performance with an accuracy of 93.2%.

Despite these efforts, it is important to note that the conventional POS tagging may not be suitable for Sanskrit. POS tagging typically assigns a grammatical or syntactic category to each word in a sentence, thereby assisting tasks like parsing and disambiguation. Unlike English, a language with a fixed word order and multiple parts of speech, Sanskrit is a free word-order language with only two primary grammatical categories: subanta and tiṅanta. All other categories are derivational and can be grouped under these two. Moreover, Sanskrit is highly inflectional, and its words carry complete morpho-syntactic information. Therefore, conducting a morphological analysis for Sanskrit would be more effective than POS tagging, given that it provides the same information in greater detail. Additionally there is also a blur between the traditional POS of nouns, adjectives and proper nouns in Sanskrit. The lack of typographical mark for proper nouns, and the fact that most proper nouns in Sanskrit are either common nouns or adjectives is a challenging impediment to address without reference to the commentaries. This would require a proper database of named-entity tailored to analyze the texts.

On the other hand, Huet [2024] introduces a concept similar to POS, proposing a set of tags designed for Sanskrit. Their article outlines the reasoning behind using a color-coding schema for segmented words in the Sanskrit Heritage Reader. This schema helps in better understanding the linguistic status of these words and can be utilized for tagging purposes.

5.2 Morphological Analysis and Tagging

Morphology is a branch of linguistics dedicated to the study of words and their constituents. According to Beesley [2004], the initial phase in computational linguistics involves the development of morphological analyzers. These analyzers leverage corpora, lexicons, morphological grammars, and phonological rules meticulously crafted by field linguists and descriptive linguists. Utilizing pre-existing linguistic resources is pivotal in constructing systems that can address a spectrum of natural language applications. Typically, morphological analysis or tagging involves assigning morphological labels to tokens in sequential data. This is a crucial step in processing highly inflectional languages like Sanskrit since they carry rich linguistic information such as gender, number, person, case, tense, mood, and other grammatical features within their intricate system of word forms. While word segmentation, morphological analysis, and compound analysis are interconnected, their ordering in this article is based on their subtle dependencies. Morphological analyzers cannot process continuous sequences of words without specific boundaries,

necessitating segmentation before morphological analysis. Therefore, word segmentation is recognized as a vital pre-processing step for Sanskrit. The discussion on compound analysis (identification of compound types) is deferred to the subsequent section as it also relies on morphological analysis in addition to mutual compatibility of the meanings.

Wide Coverage Morphological Analyzer - Bharati et al. [2006] introduce a comprehensive morphological analyzer founded on the FSA based approach, encompassing distinct modules for handling nominal (subanta), verbal (tiṅanta), derivative (kṛdanta) suffixes, and compounds. The system makes use of the Monier William's dictionary for nominal, Dhāturatnākara for verbal and Kṛdantarūpamāla for the derivative examples. With a simple Perl program, paradigms have been created for each of them. The analyzer underwent testing on various texts, including Rāmāyaṇa, Pañcatantra, segments of Harṣacarita, and basic Sanskrit readers. The study reveals that 50% of the unidentified words were attributed to typos or instances of splitting errors. Additionally, it concludes that the morphological analyzer achieves a 95% coverage on unknown texts.

Markov Model - Hellwig [2007] introduces a stochastic tagger named "SanskritTagger", which employs a Markov model for tokenizing text and conducting mophological tagging. The model incorporates a database containing lexical and grammatical dictionaries, along with analyzed Sanskrit texts. The tagging software comprises two key modules. The first module generates hypotheses about the analysis of a phrase using saṃdhi resolution and dictionary lookup. In the second module, the most probable lexical and morphological path is determined through statistical information extracted from the corpus. The study concludes by evaluating the model on five diverse Sanskrit passages of different genres and conducting error analyses. The article also discusses the challenges involved in tokenizing and tagging Sanskrit and outlines the scope for potential enhancements of the model.

JNU Analyzer - Jha et al. [2009] propose a morphological analyzer that analyzes sandhi-free inflected noun forms (subanta) and verb forms (tiṅanta). Similar to the previous approach, the modules for subanta and tiṅanta analysis are different. However, they are subsequently merged for sentence analysis. The subanta analyzer works with the help of two databases viz examples and rules. The analyzer's architecture consists of a front-end user interface and a back-end database. The database connectivity is done through the **Java Database Connectivity** (**JDBC**) driver. For tiṅanta analysis, the study makes use of two distinct tables that consist of the verb root and tiṅ termination. Thus, during analysis, the constituents are identified and matched to the existing structured database.

Morphological Analyzer for Nominal Inflections - Chandra [2012] introduces an advanced Morphological Analyzer designed for Sanskrit Nominal Inflections, employing a restructuring of Pāṇinian morphological rules for computational efficiency. The restructuring involves reordering Pāṇinian morphological rules. Approximately 1,500 Pāṇinian rules have been utilized in this study. The system has two main components: one for recognizing nominal inflections in Sanskrit texts and the other for analyzing these inflections. The proposed approach consists of three steps for analyzing Sanskrit nominal morphology: (1). recognition of all nominal inflections using relational databases (2). subsequent segmentation of the nominal word into a sequence of morphemes and stems (3). detailed information on the stem, suffix, case, and number of the nominal words. An evaluation of the system indicates an accuracy of 85%.

Neural Sequence Labeling Models - Gupta et al. [2020] focus exclusively on the efficiency of neural sequence labeling models for tagging Sanskrit texts morphologically. The four standard neural sequence labeling models used in this study are: **Monolithic Sequence Model** (**MonSeq**), **Neural Factorial CRF** (**FCRF**), **Sequence Generation Model** (**Seq**), and **Multi-task learning** (**MTL**) which is further categorized into MTL-Hierarchy and MTL-Shared based on settings. Each model employs an encoder for generating input representations, and the study utilizes a training

set of 50,000 sentences and a test set of 11,000 sentences from the DCS. The evaluation reveals that MTL-Hierarchy performs the best with an accuracy of 86.74%. However, the article also addresses the issue of syncretism[12] a notable challenge faced by all models.

All the models discussed in this section provide morphological analysis given a word. However, in most of these models, morphological analyzers form a layer for subsequent parsing tasks as stand alone implementations, and may not be very useful for real texts. The neural sequence labeling models by Gupta et al. [2020] and Byt5-Sanskrit stand out for their ability to handle real-world data, as they incorporate contextual information. In fact, the Byt5-Sanskrit lemmatizer was also tested on other morphologically rich languages, including Bulgarian, Romanian, and Turkish, where it outperformed the baseline models. As a result, Byt5-Sanskrit (refer Section 4). is also recognized as SOTA model for morphological analysis. Nevertheless, even Byt5 does not fully resolve the challenges posed by homonyms and syncretism, which remain significant issues for morphological analysis.

5.3 Compound Analysis

Compound analysis typically refers to breaking up of the compound into its constituents and understanding the relationship between them in order to make sense of the compound. In several cases compounds are formed with successive binary or n-ary combinations of smaller compounds. This structural nestedness introduces an exponential increase in the number of possible combinations, which is a catalan number C_n,

$$C_n = \frac{(2n)!}{(n+1)!\ n!}, \tag{1}$$

if there are n components in a compound [Huet 2009]. Additionally, the relation between components of the compound is not explicitly stated. Therefore, compound analysis is a challenging task. We will look at systems that have experimented with analyzing compounds. The analysis may include identification of the compounds, parsing of the compound components, and type identification of compounds.[13]

UoH Model - Kumar and Kulkarni [2010] propose a compound analysis system with four steps: 1. Segmentation, 2. Constituency Parsing, 3. Compound Type Identification, and 4. Paraphrase Generation. Segmentation and compound type identification models have been discussed in this article, while the details of constituency parsing are provided by Kumar [2012] and compound paraphrase generation is elaborated in the study by Kumar et al. [2009]. After segmentation, the system uses a constituency parser to construct a binary tree illustrating the syntactic composition of the compound in relation to the segmentation. The parser's role is to make informed decisions about how to group constituents together within the given context. The robustness of the parses hinges on the parser's ability to identify semantic compatibility among the compound components. A manual tagging process applied to 12,796 compounds achieved 72.7% precision for eight tags, though it did not consider context. The system uses "paraphrase" (vigrahavākya) expressions to convey compound meanings, with gender helping to determine compound type. Kumar et al. [2009] developed an algorithm for paraphrasing two compound types, Tatpuruṣa and Bahuvrīhi, achieving accurate paraphrases for 145 out of 160 test examples.

Grammar and Corpus-based model - Krishna et al. [2016b] propose a classifier model for compound type identification which is based on the head of the compound. This classifier model is a combination of rules extracted from the Aṣṭādhyāyī, semantic relations extracted from

[12]Inflected forms of a lemma that have the same surface form but correspond to multiple morphological tags.

[13]Compounds in Sanskrit mainly fall under four categories. They are: Avyayībhāva (indeclinable compound), Tatpuruṣa (endocentric), Bahuvrīhi (exocentric) and Dvandva (co-ordinative).

Amarakoṣa, and linguistic structural information of the data extracted using Adaptor grammar which is a probabilistic context-free grammar. The article discusses improvement in the system's performance with the inclusion of each of the above-mentioned resources. The dataset for this study was obtained from UoH and consisted of more than 32,000 unique compounds. The model was evaluated on four different ensemble-based classifier systems which are: Random Forests, **Extreme Random Forests** (**ERFs**), **Gradient Boosting Methods** (**GBM**) Adaboost Classifier. Among these, the ERF performed the best with a precision of 0.76 and an accuracy of 0.75.

Neural Model - Sandhan et al. [2019] investigate the effectiveness of neural models compared to hand-featured models to determine their performance in identifying compound types. For this, the study makes use of three different neural architectures; **Multi-Layer Perceptron** (**MLP**), **Convolution Neural Network** (**CNN**), and **Long Short Term Memory** (**LSTM**). The input to these systems is fed on various levels such as word, sub-word, and character levels, which form the different word embedding approaches. The study restricts the number of compound components to two. The text corpus used for this study includes data from DCS, as well as data scraped from Wikipedia and Vedabase. Altogether, the total number of words in the text corpus is about 4.7 M. Compared to the three different architectures used, the LSTM is reported to have performed better with an accuracy of 77.68%.

Compound classifier - Premjith et al. [2019] propose the use of a binary classifier model to distinguish between compound and non-compound words. This study employs various machine learning algorithms, including Naive Bayes, K-Nearest Neighbor, Decision Tree, Random Forest, Support Vector Machine, Multi-Layer Perceptron, Logistic Regression, and AdaBoost classifier. FastText is utilized for word embeddings in this investigation. The findings reveal that the K-Nearest Neighbor algorithm outperformed the others, achieving an accuracy of 90.38% with an embedding dimension of 500. The corpus utilized in the study consists of a tagged dataset obtained from UoH.

Multi-Task Approach - In a work by Sandhan et al. [2022a], a multi-class classification approach is proposed for **Sanskrit Compound Type Identification** (**SaCTI**). The article introduces a multi-task learning architecture incorporating contextual information for discerning Sanskrit compound types. Two auxiliary tasks, morphological tagging and dependency parsing, are integrated to enhance the overall learning of the system. The system includes a multi-lingual tokenizer to segment sentence tokens, and these segmented pieces undergo processing through a multilingual pretrained XLM-R encoder. Subsequently, the multi-task learning architecture is applied to leverage the enriched contextual information. The system outperformed the baseline models by a margin of 6.1% (accuracy) and 7.7% (F1 score). Notably, the study highlights the efficacy of the proposed architecture in handling two other languages, namely English and Marathi.

DepNeCTI - Previous compound analysis systems have predominantly focused on identifying binary (two-word) compound types. In a study by Sandhan et al. [2023], a novel system is introduced to analyze multi-component compounds. The research specifically concentrates on the recognition of nested spans within these compounds and the interpretation of their implicit semantic relationships. Given the similarity between semantic relations among compound components and dependency relations, the task is framed as a dependency parsing task, termed "Dependency-based Nested Compound Type Identification". The article introduces two new datasets, namely NeCTI and NeCTI OOD, the latter serving as an out-of-domain testbed. Standard formulations, including constituency parsing, bottom-up constituency parsing, span classifier, lexicalized constituency parsing, and seq2seq, are adapted as baselines for this task. The proposed DepNeCTI is presented in two variants–one employing LSTM and the other utilizing XLM-R. Both variants outperform the baseline systems and exhibit improvements when contextual information

is integrated. Notably, DepNeCTI-XLM-R emerges as the most effective, demonstrating significant enhancements when leveraging contextual information with a **Labeled Span Score** (**LSS**) of 83.19 for NeCTI data and 73.54 for NeCTI OOD data.

When analyzing a sample such as the Bhagavad Gita, it is observed that, on average, each verse contains at least two compounds. Additionally, Krishna et al. [2016b] report that in the DCS, about 75% of the vocabulary is made up of compounds, with 41% consisting of multiple components. This highlights the high frequency of compounds in Sanskrit, making their analysis a complex task. So far, the only model that resolves multi-component (more than two) compounds is DepNeCTI proposed by Sandhan et al. [2023] and this can be used on real data. Directing efforts toward creating more models that can handle multi-component compounds could be a valuable avenue for future work in compound analysis, as detecting compound boundaries, particularly in cases of nestedness, remains a challenging task.

6 Sentential Parsing

Parsing is the process of dissecting a sentence into smaller components and comprehending the structure and relationship among its elements. Two primary parsing approaches exist: (1). Constituency parsing and (2). Dependency parsing. Constituency parsing entails recognizing the hierarchical structure of a sentence through its constituents, particulary major phrases like **Noun Phrase** (**NP**) and **Verb Phrase** (**VP**) using productive rules such as context-free grammar. On the other hand dependency parsing involves a pair-wise direct identification of syntactic and semantic relationships between words, highlighting their role or function without consideration for the hierarchical structure of the sentence. While constituency parsing is used for languages that have a fixed word order, dependency parsing is considered more suitable for languages that have a flexible word order such as Sanskrit and other Indian languages that store syntactico-semantic information in its inflections Jurafsky and Martin [2000]. Let us go through different parsers that were designed and experimented for Sanskrit.

SH Shallow Parser - Huet [2006] presents the initial effort in crafting a parser for Sanskrit. Addressing the previously discussed over-generation issue linked to the SH segmenter, a semantic-based filtering mechanism became imperative, translating into a kāraka analysis – essentially a form of dependency parsing. This analysis entails aligning the subcategorization pattern of verbal forms, governing the crucial thematic roles essential for verbs to effectively convey meaning in expressing a situation or action. Nevertheless, the article underscores the minimal treatment of kārakas/thematic roles. The subcategorization patterns for verbs primarily categorize them as transitive or intransitive in the article. An enhancement in the analysis, encompassing the need for a dative argument in verbs related to gifting, and similar contexts, remains a potential refinement to be considered.

Parsing using Deterministic Finite Automata (DFA) - Venturing into the development of a Sanskrit parser, Goyal et al. [2007] propose a **Deterministic Finite Automaton** (**DFA**) for morphological analysis and adopts Pāṇini's "utsarga apavāda" (general and exception) methodology for relation analysis. The parser takes a Sanskrit sentence as input and dissects it using predefined rules stored in the DFA. The rule bases crafted for this endeavor comprises the nominals rule database (encompassing entries for noun and pronoun declensions), the verb rule database (comprising entries for 10 verb classes), and the particle database housing word entries. To address challenges arising from external sandhi phenomena, dedicated sandhi modules have been designed using the rule base. Upon obtaining the root word and attributes for other words, the subsequent phase involves identifying relationships among them. Rigorously tested on four distinct paragraphs, the parser has demonstrated accurate performance according to the reported results.

UoH Parser - Kulkarni et al. [2010b] explore the Sanskrit grammar to extract diverse semantic relations, aiming at formulating a grammar-based dependency parser tailored for Sanskrit and improve the model further by proposing a Deterministic Parsing algorithm [Kulkarni 2013]. The current parser, an Edge-centric Binary Joint model, is designed for both prose and verse, addressing challenges faced by the previous node-centric model, which struggled with multiple incoming arrows. This modification incorporates various concepts from Indian theories of verbal cognition (Śābdhabodha). Given that verses in Sanskrit exhibit a more flexible word order compared to prose forms, the study also presents an algorithm for converting parsed verses into their canonical prose form. This conversion is guided by insights obtained from the popular text Samāsacakra. The parser's performance was evaluated on 195 verses along with their prose counterparts. However, the parser encountered difficulty in parsing 45 instances of both verse and prose, primarily attributed to out-of-vocabulary words Kulkarni et al. [2019].

Energy Based Model (EBM) - Krishna et al. [2020b] extend the experiments on EBM conducted previously for word segmentation and morphological parsing. They define a search-based structured prediction framework using EBM for various structure prediction tasks including word segmentation, morphological parsing, dependency parsing, syntactic linearization, and prosodification for languages with free word order such as Sanskrit. This framework consists of three components viz: a graph generator, an edge vector generator, and a structured prediction model. The inputs for all the tasks are given to the graph generator by the user. The outputs from the graph and edge generator form the input for the structured prediction model. The experiment involves training of eight models, five of them in a stand-alone setting and three in a joint setting. In this study, the **Forward Stagewise Path Generation** (**FSPG**) feature function is utilized for automated learning. The article concludes that either state-of-the-art results were attained for each task, or this was the sole data-driven model experimented with for all tasks, especially prosodification which was first introduced in this article. For parsing, the article reports that the model achieved an LAS of 85.32 and a UAS of 82.65.

Data-Driven Dependency Parsing - Krishna et al. [2023] represent the initial exploration into data-driven neural dependency parsing. The experiment assesses the effectiveness of various data-driven approaches to dependency parsing, encompassing two graph-based methods and two transition-based methods. The first transition-based model, **Yet Another Parser** (**YAP**), excels in handling morphologically rich languages and employs hand-crafted features for free word-order languages. The second transition-based parser, **Learning to Search** (**L2S**), utilizes a framework that transforms structured prediction problems into search problems and incorporates hand-crafted features. The study also employs two graph-based parsers, namely **BiAFFINE** (**BiAFF**) Parser and **Deep Contextualized Self-training** (**DCST**). Both graph-based parsers automatically learn word representations as features and utilize LSTM to learn contextual embeddings. The models were trained on a dataset from the UoH, along with 1,500 sentences from the STBC. Testing involved an additional 1,000 sentences from the STBC and 300 verses, along with their corresponding prose forms, from the renowned literary work Śiśupālavadha. The evaluation was based on micro-averaged UAS and LAS. Among the four parsers, DCST exhibited the best performance with a UAS of 80.97 and LAS of 72.86 on the STBC dataset.

Dependency Parsing in a low resource setting - For morphologically rich languages, acquiring morphological tags at runtime poses significant challenges and using predicted tags from taggers can impact the performance of parsers. Addressing these issues, Sandhan et al. [2021] propose a novel approach involving simple auxiliary tasks to enhance parser performance. This pre-training strategy aims at integrating word representations from encoders trained on diverse sequence-level supervised tasks as auxiliary tasks with the default encoder of the neural parser. These tasks encompass predicting morphological tags, dependency labels, and case tags

(specifically for nominal forms). The study conducts experiments on two main neural parsers, BiAFF and DCST, with their default configurations serving as the baseline. The configuration introduced in this article, labeled as LCM (representing label, case, and morph tags), demonstrates superior performance. The findings reveal that minimally supervised pre-training yields significantly better results compared to transfer learning and multi-task learning. The article extends its experimental scope to include ten other morphologically rich languages, such as German, Hungarian, and Greek. The results demonstrate that pre-training leads to a substantial improvement, with an average absolute gain of 2 points in UAS and 3.6 points in LAS compared to DCST. The study makes use of 500, 1,000, and 1,000 sentences from the STBC as the training, development, and test data, respectively, for all models. For the auxiliary tasks, all sequence taggers undergo training using an additional set of 1,000 sentences from STBC that were not previously used.

Data-driven dependency parsing of Vedic Sanskrit - Hellwig et al. [2023] introduce the first data-driven dependency parser for Vedic Sanskrit. They use this for extending an existing treebank for Sanskrit [Hellwig et al. 2020]. The authors utilize a graph-based biaffine parser, which performs well on non-projective dependencies, along with DCST (a pre-training step) that enhances the parser. Additionally, they explore the impact of static embeddings (Word2Vec, fastText) and contextualized embeddings (RoBERTa, XLM-RoBERTa, ELMo) on the model's performance to determine which type of embeddings work best. For the pre-training steps, they use DCS data, and the Vedic Tree Bank for training the model on syntactic gold annotations. In total, around 16,272 sentences were used. The authors also report the attachment scores achieved by the proposed model: 87.61 for unlabeled and 81.84 for labeled.

Various approaches have been ventured for Sanskrit dependency parsing.[14] While rule-based parsers demonstrate proficiency, they fail when dealing with complex compounds. On the other hand, neural parsers encounter challenges due to limited training resources and the utilization of morphological information to predict tags during runtime. Developing parsers capable of addressing these challenges is necessary to deal with real world data. One major challenge for sentential parsing in Sanskrit text is the absence of punctuation that makes it challenging to define sentence boundaries. The daṇḍa, commonly used in poetry, serves more as a prosodic marker than a syntactic one. As a result, sentence boundaries are often ambiguous, extending beyond a single śloka and occasionally running across entire pages, as seen in works like Kādambarī. Despite its importance, little research has been dedicated to this area.

Although some of the neural parsers discussed in this section seem efficient to handle real data, with some even reporting good performance on other morphologically rich languages, Byt5-Sanskrit (refer Section 4) surpasses the performance of all the models. It shows great performance in all the three tasks (segmentation, lemmatization and parsing) individually, as well as jointly. Thus, we report this to be SOTA for Sanskrit dependency parsing.

Apart from these efforts, Arjuna and Kulkarni [2016] have worked on segmentation, analysis and graphical representations of Navya-Nyāya expressions. The Navya-Nyāya school of Indian logic and philosophy has developed a sophisticated language to address verbal cognition, logic, and epistemology and is known for its use of long compounds. The authors design a tool that includes a domain-specific segmenter, a semi-automatic constituency parser, and a context-free parser that translates the Navya-Nyāya expressions into a conceptual graph. This effort is very helpful in understanding better the scholastic Sanskrit commentaries that uses complex compounds using the relational notation as per the Navya-Nyāya concepts. However, this still remains a preliminary effort and more concrete work is to be done in this direction.

[14]See Table 2 for a brief summary.

Table 2. Parser

Sl.No	Model	Year	Type	Data
1	SHR Shallow Parser	2006	Rule-based	-
2	DFA Parser	2007	Rule-based	-
3	Unsupervised Parser	2009	Statistical	Sanskrit Tagger
4	Parser for Verses (UoH)	2019	Rule-based	-
5	EBM	2020	Neural	DCS
6	Data-driven Parser	2020	Neural	UoH Corpus
7	Jivnesh	2021	Neural	Sanskrit Treebank Corpus
8	Byt5-Sanskrit	2024	Pre-Trained	Sangraha and DCS

7 Discourse Parsing

This section discusses the application of discourse level parsing, such as anaphora resolution and question-answering.

7.1 Anaphora Resolution

Anaphora is a linguistic phenomenon where an expression is used to refer back to an item within the context. The term "anaphor" is used to describe the expression pointing back, while the entity to which it refers is known as the "antecedent" [Mitkov 1999]. Anaphora resolution involves computationally analyzing which anaphor corresponds to which antecedent. This task has been extensively studied in NLP. Similar to other languages, Sanskrit also presents challenges in dealing with anaphors, making their resolution a complex task. Despite the widespread exploration of anaphora resolution in various languages, there has been limited exploration for it in Sanskrit.

Sanskrit Anaphora - There are various types of anaphora, including pronominal anaphora, noun phrase anaphora, demonstrative anaphora, reflexive anaphora. Pralayankar and Devi [2010] develop an algorithm specifically designed to identify pronominal anaphora in Sanskrit. In the process of anaphora resolution, determining clause boundaries becomes crucial, as the position of the noun significantly influences the identification of the antecedent. Consequently, the preprocessing stage begins with the delineation of clause boundaries and the identification of subjects and objects using a morphological analyzer. Subsequently, NP preceding the **Pronouns** (**P**) are identified. The NP that matches in **Person, Number, and Gender** (**PNG**) with the pronoun is then recognized as the antecedent according to the algorithm. The algorithm was tested for 200 sentences of which 40 had pronominals. However, the system could correctly identify only 26 of the 40 pronominals.

SARS - In the study by Gopal and Jha [2014], an automatic resolution system for anaphors and cataphors in Sanskrit is discussed. The system utilizes POS tagged data to address lexical anaphors. The initial processing involves sentence tokenization and word tokenization of the input data. The system operates on one tokenized sentence at a time in the resolution of anaphors. Specifically, it examines each sentence for lexical anaphors, such as reflexive and reciprocals, and proceeds to the next sentence if none are found. When anaphors are identified, the system employs an algorithm to search for potential antecedents and resolves them. The candidate antecedents considered by the system include **common nouns** (**NC**), NP, **personal pronouns** (**PPR**), and **relative pronouns** (**PRL**). These categories are distinguished based on their morphosyntactic tags, underscoring the importance of POS tagging information in the anaphora resolution process. The system's performance was evaluated on 3659 sentences, revealing that out of 115 anaphor occurrences, only 70 were correctly identified by the system.

7.2 Question Answering (QA)

Question Answering is defined as the task through which human-posed questions in natural language are automatically answered. QA systems prove particularly beneficial when a user requires highly specific information and either lacks the time or inclination to go through all the available documentation related to the search topic to address the current problem [Mollá and Vicedo 2006]. There are different types of QA systems. They are:

(1) Systems based on search engines to retrieve answers.
(2) Systems that make use of linguistic information and machine learning methods to extract answers.
(3) Systems that extract answers from structured data sources such as **Knowledge Base** (**KB**).
(4) A hybrid model that consists of all three [Soares and Parreiras 2020].

Although the advancement of QA systems has demonstrated significant benefits across various domains, there has been limited exploration in the development of QA systems for Sanskrit. The sole implementation within this domain is detailed below.

Question-Answering in Sanskrit through Automated Construction of Knowledge Graphs - Terdalkar and Bhattacharya [2023a] propose the development of a natural language question-answering system for Sanskrit, employing KB to store information in the structured format of **Knowledge Graphs** (**KG**). These graphs utilize semantic triplets (subject, object, and predicate) to encode relationships and the knowledge extraction process automatically generates triplets. In this study, two monumental texts, Rāmāyaṇa and Mahābhārata, are utilized for capturing human relationships. Additionally, an Ayurvedic text, Bhāvaprakāśa Nighaṇṭu, is employed to identify synonymous relationships, emphasizing the significance of technical texts. The primary focus of the study revolves around identifying terms representing relationships. The subsequent crucial task involves parsing natural language questions, utilizing a similar triplet pattern. Once this parsing is accomplished, a SPARQL query pattern is constructed to interact with the KG, facilitating the retrieval of answers. The research assesses the system's performance across three distinct tasks: question parsing, conditional question answering (evaluating correctness), and the comprehensive question-answering task. A set of 35 questions from Rāmāyaṇa and 45 from Mahābhārata, sourced from 12 different users, formed the basis of the evaluation. The system yielded a precision score of 0.55 for the overall question-answering task when considering both texts collectively. Further for Bhāvaprakāśa Nighaṇṭu, the task was to identify a śloka synonymous with another. This was accomplished using the word vectors obtained for each śloka. With differing train and test data sizes in four different scenarios, different accuracy scores were obtained. However, the article concludes that the accuracy scores for all the components could be improved.

8 Machine Translation

Machine Translation (**MT**)[15] stands out as a key application of NLP, facilitating the seamless transfer of information across diverse regions of the world. Machines play a pivotal role in rapidly translating extensive data from one language to another. While Sanskrit may not be a widely spoken language today, it holds a wealth of ancient Indian knowledge systems that pique the interest of many. The contribution of machines in assisting to unravel this valuable knowledge is highly significant. Although the attempts to create good translation systems for Sanskrit are numerous, yet the unique grammatical structure and rich contextual nuances of Sanskrit pose intricate challenges that continue to be a focal point in the quest for more accurate and nuanced translations.

[15]Since MT is an application of both sentential and textual parsing, this has been considered as a separate section.

8.1 English to Sanskrit

Experiments on MT for Sanskrit commenced in 2007 with Goyal and Sinha [2007] attempting to modify the AnglaBharati system design and adapting it for Sanskrit. The goal was to translate basic sentences from English to Sanskrit based on the **Pseudo Lingua for Indian Languages** (**PLIL**), which is similar to context-free grammar. A text generator was constructed for the Sanskrit language, employing Pāṇinian grammar principles to generate Sanskrit translations. This was followed by a rule-based system introduced by Mane et al. [2010]. In this approach, the input (English sentences) is tokenized and subjected to morphological analysis. Through a bilingual dictionary, source words are then mapped to their equivalents in the target language. The reordering of words, guided by the grammar rules of the target language, produces the final output. The sentences were then converted into a waveform through which the audio was generated.

It is important to acknowledge that both these experiments were conducted on a relatively small scale. However, they laid the foundation for more comprehensive and sophisticated systems that emerged subsequently. As technology advanced and understanding of the complexities of Sanskrit improved, full-fledged systems with enhanced capabilities and broader applications emerged. They are discussed below:

ANN and Rule-based Model - Mishra and Mishra [2010] introduce a prototype model to translate from English to Sanskrit, combining traditional rule-based machine translation with an **Artificial Neural Network** (**ANN**) model. While the ANN model identifies equivalent Sanskrit words for English sentences, the rule-based model generates Sanskrit translations based on the rules governing noun and verb formation. The system employs a feed-forward ANN to select Sanskrit words from the English to Sanskrit **User Data Vector** (**UDV**). Given the morphological complexity of Sanskrit, the system relies minimally on syntax, using morphological markings to recognize Subject, Object, Verb, Preposition, Adjective, Adverb, and Conjunctive sentences. The system can only handle English sentences that are of the forms: (i) simple subject, object and verb; (ii) subject, object, adverb and verb; (iii) subject, object, adjective and verb; (iv) subject, object, preposition and verb; (v) interrogative sentences and (vi) compound sentences. Evaluation metrics such as BLEU and METEOR are employed, and the article concludes by presenting scores for twenty randomly selected sentences.

Transfer-based - Pathak and Godse [2010] introduce in their article a transfer-based approach for translating from English to Sanskrit. The fundamental concept behind Transfer-based Machine Translation is that the input sentence can be converted into the output sentence by simplifying the parsing structure of the source language into an equivalent parse structure in the target language. This process requires essential grammar rules for both the source and target languages. Once the parse structure of the target sentence is formed, each word in the parse tree structure is searched for its meaning in a bilingual dictionary.

EtranS - Bahadur et al. [2012] introduce a comprehensive framework called "EtranS" designed for translating a series of English sentences into Sanskrit. The model incorporates a set of translation rules to convert source sentences into target sentences. The translation process involves breaking down the source sentence, extracting information from a dictionary, validating the information based on predefined rules, searching for compatible target words, translating, and ultimately generating the output. The English-to-Sanskrit translation model relies on the formulation of **Synchronous Context Free Grammar** (**SCFG**), a subset of **Context Free Grammar** (**CFG**), for linguistic representation of syntax. The software is designed with a focus on simplicity and effectiveness, particularly addressing the grammar of both the source and target languages. The results indicate a successful translation rate of ninety percent for the sentences, with a two percent reported error rate attributed to linguistic ambiguities.

8.2 Sanskrit to English

Interlingua-based - In an interlingua-based MT approach, the focus is on creating a language-independent structure, or interlingua, to aid translation. It emphasizes that machine translation should go beyond syntax and semantics to include a deeper understanding of text content. Therefore, Sreedeepa and Idicula [2017] use the Pāṇinian framework as the foundational structure and propose a translation system. The systematic process for translating a Sanskrit sentence begins with tokenization, followed by the generation of feature structures for the individual tokens. Subsequently, interlingua is constructed by applying kāraka analysis rules to the feature structures developed for the sentence. The next step involves mapping and generating target language tokens based on the established interlingua, followed by the creation of f-structures for these target language tokens. The final translation output in the target language is then generated by applying rules to the f-structure of the target language tokens. The system was tested on, approximately 35 sentences where it demonstrated accurate outputs for all instances. Performance evaluation was conducted using two metrics: human-based evaluation, utilizing scales such as CCOR, MEAN NONS, and DANG, and automatic metric evaluation, with BLEU selected as the benchmark for assessing translation quality.

Reinforcement and Transfer Learning Approach - Punia et al. [2020] explore novel approaches previously unexplored in the translation of Sanskrit to English. Initially, the study establishes a baseline using the Transformer architecture, incorporating a multi-head self-attention mechanism within an encoder-decoder framework. Followed by this, advancements to the transformer model involve the integration of Reinforcement Learning and Transfer Learning, resulting in a reported enhancement in BLEU scores attributed to the Transfer Learning mechanism. In addition to these methodological contributions, the study enriches the field by providing an open dataset comprising both parallel and monolingual data. The parallel dataset, totaling 9,000 lines, is sourced from diverse sources, including 2,100 sentences from OPUS, Sanskrit translations of the Bible, Shlokas from the Rāmāyaṇa, and additional sentences from the Bhagavad Gītā. The monolingual Sanskrit data consists of approximately 130,000 lines, utilizing the Romanized version of the Mahābhārata. For English monolingual data, the study extracts content from the Europarl dataset.

RNN Model - Koul and Manvi [2021] propose a neural machine translation system for translating Sanskrit to English, employing **Recurrent Neural Networks** (**RNNs**). The model uses vectors representing the source Sanskrit sentence, the target English sentence, and a bilingual dictionary. The RNN uses a computational graph to analyze words of the sentence in parallel, breaking them down into their roots and linguistic categories. The dependencies of the analyzed words concerning the context are determined and stored in the vector for subsequent processing. For simple sentences, words are translated using the bilingual dictionary, while for complex compounds, the learner module suggests translations. Partial sentences are generated, and context is used to adjust grammatical aspects like tense. Any additional extra-linguistic information if present is incorporated and finally the output is generated. The model employs Naive Bayes and Support Vector Machines in the classification algorithm to identify the most suitable target word. Notably, the study highlights that the use of a Support Vector Machine classifier in determining the appropriate target word, yielded higher accuracy compared to the Naive Bayes approach.

Hybrid model - The work by Sitender and Bawa [2021] introduce a **machine translation system** (**MTS**) designed for translating Sanskrit to English that employs a hybrid approach, combining both direct and rule-based translation techniques. The system addresses language divergence among Sanskrit and English languages and proposes a solution to handle it. The MTS incorporates two bilingual dictionaries, a tagged Sanskrit corpus, a Sanskrit analysis rule base, and an **English Language Grammar Rule** (**ELGR**) base. It comprises of six modules that include source language

preprocessing, database, Sanskrit grammar demonstration, English parse tree generation, translation using the direct method, and output generation. For Sanskrit language processing, the system utilizes CFG in **Chomsky Normal Form** (**CNF**) and employs the **Cocke–Younger–Kasami** (**CYK**) parsing technique for processing input Sanskrit sentences. The article also introduces a novel algorithm that constructs a parse tree from the parsing table. The target language sentence is generated through the combined efforts of the ELGR base and bilingual dictionaries. Evaluation of the proposed system was conducted using the Natural Language Toolkit API in Python, resulting in a BLEU score of 0.7606, a fluency score of 3.63, and an adequacy score of 3.72.

Itihāsa - Aralikatte et al. [2021] introduce a large-scale translation dataset containing 93,000 pairs of Sanskrit shlokas and their English translations extracted from two famous Indian epics, the Rāmāyana and the Mahābhārata. The abridged versions of these two texts authored by M.N. Dutt have been taken acknowledging its large size and popularity. Further, this study attempts to highlight the complexity of the dataset by training and evaluating the performance of standard translation models. This includes one statistical model and five neural models. For SMT Moses is used and for NMT seq2seq Transformers with standard BERT architectures has been used. The evaluation is based on four parameters (i) character n-gram F-score (ii) token accuracy (iii) BLEU score and (iv) Translation Edit Ratio score. The study concludes that all the models performed poorly with low token accuracy and high translation edit ratio and underscores potential areas for improvement in the translation models.

8.3 Sanskrit and Other Indian Languages

Several efforts have been made to translate Sanskrit to other Indian languages and vice-versa. Although we will not be discussing them in detail, we will be mentioning a few of them, acknowledging their contributions.

The initial system implemented for Sanskrit-Hindi MT was at the Jawaharlal Nehru University, Delhi (Special Centre for Sanskrit Studies) [Pandey and Jha 2016]. This system was trained on two different platforms which are **Microsoft Translator Hub** (**MTHub**) and Moses with around 35000 sentences. Subsequently, multiple **Neural Machine Translation** (**NMT**) models were experimented with for the Sanskrit-Hindi language pair. Singh et al. [2019] introduce a hybrid system that combines NMT and RBMT, achieving a BLEU score of 61.2% and an accuracy of 99%. Following this Kumar et al. [2019] propose a Zero-Shot Translation method trained on English-Hindi and Sanskrit-English language pairs. A corpus-based NMT was presented by Singh et al. [2020], marking the first of its kind for the Sanskrit-Hindi language pair. This corpus-based model, incorporating a deep neural network, divides the workflow into three phases: (i) Data Analysis, (ii) Data Processing, and (iii) Target Generation. Another NMT model, employing an encoder-decoder with self-attention and the inclusion of gated recurrent units, was experimented with by Sethi et al. [2023] for the Sanskrit-Hindi language pair. This system outperformed all existing translation systems for the same language pair. Additionally, an RBMT system was implemented for translation from Hindi to Sanskrit, leveraging the crucial linguistic features of both languages. Efforts were extended to translate Sanskrit into Gujarati and Malayalam as well. Raulji et al. [2022] proposes a grammatical transfer approach for translating Sanskrit to Gujarati. To enhance implementation accuracy and translation clarity, in-depth research on the creation of Nouns, Verbs, Pronouns, and Indeclinables, along with their mappings, was conducted. Chingamtotattil and Gopikakumari [2022] introduce an NMT for Sanskrit-Malayalam, utilizing an attention-based mechanism for its development. The study further improves the system by incorporating morphological elements and evolutionary word sense disambiguation.[16]

[16] A brief summary of all MT tasks has been presented in Table 3.

Table 3. Machine Translation

Sl.No	Model	Year	Type	Source to Target
1	AnglaBharati	2007	Rule-based	English to Sanskrit
2	Deepak	2009	Rule-based	English to Sanskrit
3	Anusaarak	2009	Rule-based	Sanskrit to Hindi
4	ANN	2010	Rule-based	English to Sanskrit
5	Transfer-based	2010	Rule-based	English to Sanskrit
6	EtranS	2012	Rule-based	English to Sanskrit
7	JNU	2016	Statistical	Sanskrit to Hindi
8	Interlingua-based	2017	Rule-based	Sanskrit to English
9	RNN	2017	Neural	Sanskrit to English
10	RBMT and NMT	2019	Neural	Sanskrit to Hindi
11	Zero-shot	2019	Neural	Sanskrit to Hindi
12	Hybrid	2020	Rule-based	Sanskrit to English
13	Reinforcement and Transfer Learning Approach	2020	ML	Sanskrit to English
14	CB DNN	2020	Neural	Sanskrit to Hindi
15	Itihāsa	2021	Neural	Sanskrit to English
16	Transfer-based	2022	Rule-based	Sanskrit to Gujarati
17	NMT	2022	Neural	Sanskrit to Malayalam
18	NMT	2023	Neural	Sanskrit to Hindi

In addition to the aforementioned efforts, the widely used and easily accessible platform for translations between Sanskrit and various other languages is the Google Translate.[17] In 2022, Google Translate, which uses the Transformer architecture, expanded its language repertoire to include Sanskrit.[18] The inclusion of Sanskrit in Google Translate marked a significant milestone, playing a pivotal role in expanding the accessibility and reach of the Sanskrit language. Although there have been no quantitative studies analyzing Google Translate's performance specifically for Sanskrit, a preliminary study by Pradeep et al. [2024] examined its qualitative performance. This study focuses on word meaning errors in the English translation of the Bhagavad Gita and highlights how the performance of Google Translate is affected when dealing with polysemy.

9 Literary Analysis

This section discusses tasks that are specific to the analysis of poetry (kāvya) in Sanskrit. Sanskrit poetry is divided into śravyakāvya (that which is heard: poetry and prose), and dṛśyakāvya (that which is seen: drama). Śravyakāvya is further divided into gadya (prose), padya (poetry) and campu (amalgamation of poetry and prose). Despite its divisions, Sanskrit poetry on a whole presents a wide range of features such as rasa (sentiment), dhvani and vyañjana (suggestion), alaṅkāra (figures of speech), chandas (prosodical meter), vakrokiti (expression) and more. Among these, preliminary efforts have been made to computationally analyse a few of them. Besides this, the current section also discusses prose order normalization which is yet another major task pertaining to Sanskrit poetry.

9.1 Chandas Analysis

A substantial portion of Sanskrit literature is composed in verse rather than prose, and these verses adhere to specific metrical patterns referred to as "Chandas" [Rama and Lakshmanan 2010]. There are different kinds of chandas in Sanskrit and the scholarly pursuit of analyzing and ascertaining

[17] https://translate.google.co.in/
[18] https://blog.google/products/translate/24-new-languages

the metrical structure of a verse assumes an important role in comprehending the poet's stylistic preferences and poetic prowess. Nevertheless, the task of metrical identification proves to be a nuanced and intricate endeavor. In response to this scholarly exigency, efforts have been undertaken to develop computational tools to automate the metrical classification process. They have been discussed below.

Meter Identifying Tool (MIT) - Melnad et al. [2013] have developed an automated tool for identifying metrical patterns, building upon the earlier work of Anand Mishra.[19] The tool encompasses a meter database containing metrical definitions presented in a machine-readable text file, and utilizes the Sanskrit Library Phonetic Basic encoding (SLP1) for processing. Both the input and display are in SLP1 format. Accepting either a single line or the entire verse as input, the tool yields a comprehensive output comprising the meter's name and type linked to its definition, the parsed string represented in orthographic syllables, the scansion (prastāra) depicting the arrangement of light (laghu) and heavy (guru) syllables, the pattern of gaṇas, and the count of syllables or mātrās. To validate its efficacy, the tool underwent testing on 1,031 verses from Pūrṇabhadra's Pañcākhyānaka. Notably, MIT achieved a remarkable accuracy rate of 98.6%, accurately identifying the meter type for 1,018 out of the 1,031 verses.

A user-friendly tool for metrical analysis of Sanskrit verse - An approach very similar to the previous one has been discussed by Rajagopalan [2020]. The current system differs from the previous one in the following ways:

(a) It accepts input in several Roman transliteration schemes such as **International Alphabet of Sanskrit Transliteration** (**IAST**), **Harvard-Kyoto (HK)**, and **Indian Languages Transliteration** (**ITRANS**), Unicode Devanāgarī, and Unicode Kannada scripts and then converts it to SLP1.
(b) The metrical database is not used as it is, instead, it is converted into data structures through which indices are generated. The indices are called pāda1, pāda2, pāda3, pāda4, ardha1, ardha2, and full.
(c) The user can also look up for partial and incorrect verses. The system generates all potential matches for the user input.
(d) The tool not only helps in identifying the type of meter but also identifies errors if any in a given verse. For instance, in the first version of the tool, the first meter added was Mandākrāntā and the entire Meghadūta text (which is completely composed in Mandākrāntā) was given as the input to the tool. The tool successfully identified 23 errors out of the 122 verses. The tool also provides a link, where the user feedback is taken.

Skrutable - Continuing the line of previous research, Tyler [2023] proposes a new tool for meter type identification called Skrutable. Although Skrutable shares similarities with the previously discussed tools, there are a few noteworthy distinctions. In contrast to the JavaScript-based nature of the other two systems, Skrutable is implemented in Python and supports a broader range of notations, including Unicode-based IAST, ASCII-based SLP1, HK, ITRANS, WX, Unicode-based Devanagari, Gujarati, and Bengali. Skrutable operates on a "cascade" of checks using fixed regular expressions and patterns. Therefore, it does not recognize verses with fewer or more than four quarters, meaning it does not support partial verses. A key feature of Skrutable is its continuous pāda resplitting and cascade-based testing until a perfect meter match is found. All potential results, including partial matches, are collected and scored based on an empirically calculated look-up table. The tool then reports the match with the highest score as the best result. The tool was tested on a dataset comprising 150,000 verses, including Prakrit, sourced from GRETIL and the repository of

[19]Currently not available.

Vishwas Vasuki.[20] It was evaluated based on four parameters: accuracy, usefulness, speed, and ease of use. The article concludes by highlighting that the analysis of meters for the entire Rāmāyaṇa text was accomplished within 14.5 seconds.

Chandojñānam - Terdalkar and Bhattacharya [2022] introduce a web-based system for meter identification and utilization called Chandojñānam, that not only accepts plain text and text files but also accommodates images as input. The tool recognizes various transliterations detected by indic-transliteration and processes images from Google OCR and Tesseract OCR engines. Chandojñānam follows a workflow similar to previous tools, emphasizing meter identification. However, it distinguishes itself by focusing on tolerance for erroneous text and providing mechanisms for text correction. The system underwent evaluation using three versions of Meghadūta from Wikisource, sanskritdocuments.org, and GRETIL. Additionally, three other texts, namely Śāntavilāsa, Śrīrāmarakṣāstotra, and Rājendrakarṇapūra, featuring various meters, were extracted from Wikisource. This compilation formed a dataset comprising 1,038 verses with 17 distinct meters. In a comparative analysis, Chandojñānam, along with the tools developed by Rajagopalan [2020] and Tyler [2023], was evaluated on the same corpora. Chandojñānam demonstrated a high accuracy of 98.2% in detecting the correct meter from erroneous verses.

9.2 Alaṅkāra Analysis

Vāmana defines "alaṅkāra" as beauty (saundaryamalaṅkāraḥ) [Raghavan 1942]. The term "alaṅkāra" in Sanskrit poetry signifies beauty that is derived from embellishment or adornment. Such an adornment or beautification can be of both its form (language) and content (meaning). In the realm of aesthetics (alaṅkāraśāstra), the rhythmic arrangements and alliterations that enhance the beauty of words are referred to as śabdālaṅkāras. Figures of speech, such as simile, metaphor, and personification, contributing to the aesthetic appeal of the content or meaning of the text, are referred to as arthālaṅkāras [Raghavan 1942]. Recent efforts have been initiated toward computationally identifying alaṅkāras in Sanskrit, with limited work discussed thus far.

Yamaka Identifier - Yamaka is a type of śabdālaṅkāra, wherein a specific group of phonemes is deliberately repeated within a stanza, contributing to a distinctive rhythmic pattern. Barbadikar and Kulkarni [2024] explore a tool designed for identifying and categorizing Yamakālaṅkāra based on the definitions provided by three famous rhetoricians: Bharata, Daṇḍin, and Rudraṭa. The system accepts input in various formats, and the internal processing is facilitated using WX notation. Subsequently, n-grams are generated for each half of the verse and sorted independently, followed by a positional check for repetition and subsequent classification. In testing, the tool demonstrated accuracy by correctly identifying and classifying 163 out of 185 verses.

Anuprāsa Identifier - Similar to Yamaka, Anuprāsa is also a form of śabdālaṅkāra, involving the repetition of consonants in close proximity. The renowned rhetorician Viśvanātha categorizes Anuprāsa into five types which are discussed thoroughly in the study by Barbadikar and Kulkarni [2024]. This study aims at identifying and classifying Anuprāsaālaṅkāra based on Viśvanātha's classification. The implementation of this tool closely resembles the Yamaka identifier. The tool initially transforms of all nasals into anusvāra to avoid ambiguities based on spellings. Then the algorithm based on the five-fold classification is applied. The tool was tested on 70 ślokas from śāstric texts and Raghuvaṃśa, along with 10 prose passages from Bāṇabhaṭṭa's Kādambarī. The study concludes that the tool effectively handles Anuprāsa instances and classifies them into defined categories.

[20] https://github.com/sanskrit/raw_etexts

9.3 Prose Order Normalization

The arrangement of words in Sanskrit verses are in general not aligned with its verbal cognition, i.e., the word order is jumbled such that they adhere to one of the Sanskrit meters mentioned in the prosodical framework. Therefore, reorganizing their structure into a prose is a very important task that requires linguistic expertise. A challenge that Sanskrit shares with other inflectional languages like Greek and Latin is the absence of clearly parenthesized phrase structures, which often leads to dislocated sentences. To address this, prose order normalization is an essential task, though it has not been extensively explored by existing systems. In many cases, this is treated as a sub-task in parsing, however Krishna et al. [2019] present kāvyaguru, a linearization approach for this task, which involves two pretraining steps and a seq2seq model with CNNs. The first pretraining step learns task-specific token embeddings, while the second uses self-attention to generate multiple word order hypotheses. These hypotheses are then processed by the seq2seq model. Trained on verses from the Rāmāyaṇa, the model achieves a BLEU score of 55.26, outperforming baseline models.

10 Citation Matching

Word matching is of great importance among tasks pertaining to information retrieval. It involves considerations on digital library representation and management, an area that remains underdeveloped so far. Even within word matching, the sub-task of citation matching remains quite unexplored for Sanskrit. Citation matching is closely related to alignment of manuscripts and philological processing like critical editions. In literature, it typically involves determining the occurrences of citations within a given textual corpus [Prasad and Rao 2010]. Sanskrit literature is rich in inter-textuality in the various categories of commentaries such as vṛtti, vākya, ṭīkā, locanā, dīpikā, and so on. For instance if we take Pāṇini's Aṣṭādhyāyī, which is the base text. We have Kātyāyana's vārtikas followed by Patañjali's Mahābhāṣya as the primary commentaries. Then we have several other commentaries such as Siddhāntakaumudī, Kāṣikāvṛtti, Tattvabodhinī, Bālamanoramā and so on. This poses a challenge to digital libraries in representing the successive hierarchical versions of the commentaries. While there have been numerous citation-matching studies for other scientific texts, Sanskrit texts have been largely overlooked due to these challenges posed by them. Nevertheless, we do have a study by Prasad and Rao [2010] addressing this gap. This study presents citation matching methods for Sanskrit through an illustration of matches between Mahābhārata-Tātparyanirṇaya (as the text containing citations) and the original Mahābhārata (as the source text to search for citations). The study makes use of two citation matching techniques which are: exact citation matching and approximate citation matching. The article explores algorithms for both exact and approximate matching, highlighting the challenges of exact matching in Sanskrit, particularly considering the presence of pāṭhāntaras (different readings) of the same corpus. While the study provides insights into cutoff results for approximate matching, it falls short in examining human evaluation for both citation-matching methods.

11 Aṣṭādhyāyī Simulation

Pāṇini's Aṣṭādhyāyī serves as a logical framework for the Sanskrit language. The aphorisms (sūtras) within this grammatical treatise intricately delineate the structure of Sanskrit in terms of phonology, morphology, and syntax. The aphorisms of Aṣṭādhyāyī can be grouped under six categories.[21] They are:

[21] saṃjñā ca paribhāṣā ca vidhirniyama eva ca |
atideśo'dhikāraśca ṣaḍvidham sūtralakṣaṇam ||

(1) saṃjñā sūtras: naming rules that assign labels which are subsequently utilized in other sūtras.
(2) paribhāṣā sūtras: meta-rules that govern the interpretation and applicability of sūtras.
(3) vidhi sūtras: prescribing rules that handle addition, deletion and insertion.
(4) niyama sūtras: conditioning rules that further constrain the application of vidhi sūtras.
(5) atideśa sūtras: extension rules that portray extension in other contexts
(6) adhikāra sūtras: governing/division rules that segment content and extend meaning to upcoming sūtras.

Sridhar and Srinivasa [2008] state that Aṣṭādhyāyī can be conceptualized as an automaton designed to generate words and sentences. The entire Aṣṭādhyāyī is structured such that general rules are initially presented, followed by their corresponding constraint rules. The systematic and mathematical arrangement of Aṣṭādhyāyī has intrigued numerous linguists and computer scientists, sparking interest in computationally modeling it such that words and sentences are generated automatically according to the rules of Pāṇini. Despite several theoretical works delineating the computational aspects of Aṣṭādhyāyī, fewer efforts have been made to practically implement these discussions.

The initial attempt to simulate Pāṇini's Aṣṭādhyāyī was undertaken by His Holiness Dr Shivamurthy Swamiji, who developed a unique software known as "Ganaka Ashtadhyayi". Followed by this Mishra [2007] in his article, mapped the rules of Aṣṭādhyāyī to mathematical equations and implemented a simulator called **Pāṇinian Sanskrit Simulator** (**PaSSim**). The primary objective of his study was to construct a lexicon based on Pāṇini's framework. PaSSim comprised five modules: database, grammar, templates, FSA, and display. The program aimed at providing a step-by-step derivation of an input word. Following this work, Mishra [2009] presented another study that enhanced the previous model. The model features an analytical phase using heuristics and an improved reconstitution phase based on Aṣṭādhyāyī rules. Its goal is to infer the constituent elements of an expression through improvised heuristics. The model, supported by a custom database, includes four modules: input, analyzer, synthesizer, and output. The article also explores the circular nature of the grammatical process, from a provisional expression to the final Sanskrit expression. This progression is followed by an extension to the existing model through the incorporation of Vedāṅgas.[22] Mishra [2010] introduces a broader coverage of the Pāṇinian meta-language incorporating a mechanism for the automatic application of rules and situating the grammatical system within the procedural complexes of Vedāṅgas. The synthesizer module, as presented in the preceding study, mirrors the general model designed to represent the fundamental processes of the Vedāṅgas.

Apart from these efforts by Mishra, we also have Goyal et al. [2009] who discuss the programming perspective of Aṣṭādhyāyī, specifically examining the order of its rules. The focus is particularly on three prominent aiding aphorisms: "pūrvatrāsiddham", "asiddhavad atrābhāt", and "ṣatvatukor asiddhah". The study not only explores the theoretical aspects but also implements Aṣṭādhyāyī using multiple modules, delving deep into its nuances. Additionally, Sridhar and Srinivasa [2008] and Subbanna and Varakhedi [2010] have undertaken the task of establishing a mathematical framework for the computational analysis of Aṣṭādhyāyī. While the former elaborately focuses on the structure of Aṣṭādhyāyī and conflict resolution techniques present in Aṣṭādhyāyī as well as the Vārtika's of Kātyāyana, the latter mathematically maps the concept of asiddhatva in Aṣṭādhyāyī to a new concept called filter. Further an XML formalization of the Aṣṭādhyāyī that can be converted to executable code was given by Scharf [2015]. A recent article by Prasad [2024], introduces a quick word generator program known as "vidyut-prakriya" based

[22]The six auxiliary disciplines that support the understanding of Vedas along with their preservation.

on the Aṣṭādhyāyī. Vidyut-prakriya implements more than 2,000 rules of the Aṣṭādhyāyī supporting tiṅantas, kṛdantas, taddhitāntas, and subantas, with partial support for samāsas and accent. Around 500 rules from the Uṇādipāṭha and around 50 rules from the Pāṇinīyaliṅgānuśāsanam have also been implemented as a part of the program. The main aim of this study was to create a finite program that could potentially generate infinite word forms in Sanskrit in a fast and feasible manner. Notably, through informal benchmarks, the study indicates that the vidyut-prakriya program is almost three orders of magnitude faster than comparable opensource systems.

There have also been efforts to simulate portions of Aṣṭādhyāyī for word generation tasks. Jha [2004] proposes a model designed for constructing nominal inflectional morphology in Sanskrit. This model is implemented using PDC-32 Prolog, and for each input, the program generates 24(21+3) śabdarūpa (wordforms) along with parsed constituents, providing detailed information. It is worth noting that while this study draws on Aṣṭādhyāyī, it also incorporates Siddhānta Kaumudī, an explanatory text on Aṣṭādhyāyī that follows a distinct order of the aphorisms. In their article, Satuluri and Kulkarni [2013] introduce compound generation, referencing the rules outlined in the Aṣṭādhyāyī concerning compound formation. The article presents this process as a phrase structure grammar, wherein the sūtras that denote technical terms are represented as context-free rules, and the vidhi sūtras are depicted as regular expressions. Similarly, Krishna and Goyal [2015] propose an object-oriented model that automatically generates derivative nouns (Taddhitas). The rules of Aṣṭādhyāyī are modeled as classes and then grouped based on the concept of anuvṛtti (a carry-forward methodology followed by Pāṇini). In addition to generating the taddhita forms, the model adheres to the sequence of rules applied as per Aṣṭādhyāyī in the derivation process, along with the appropriate usage of conflict resolution techniques needed in the selection of rules. The model was evaluated by five Sanskrit linguists on 60 sample cases where a total of 10 cases were evaluated as incorrect. Patel and Katuri [2015] introduce "Prakriyāpradarśinī", a system that imitates subanta derivation process. This model makes use of the prakriyā method given in Siddhāntakaumudī of Bhaṭṭojī Dīkṣita and not Aṣṭādhyāyī directly. In the article, an optimum order of application of rules from Sanskrit NLP perspective has been ventured and an "NLP order model" and "NLP order hypothesis" has been proposed for coding subantaprakaraṇa (chapter that discusses nominals) of the Siddhāntakaumudī.

12 Digital Infrastructure for Sanskrit

We discuss the digital infrastructure for Sanskrit under two broad categories: computational platforms, digital libraries and repositories used for training software.

12.1 Computational Platforms

Several platforms facilitate the computational analysis of Sanskrit in a web-interactive mode while also providing resources for such analysis. They are discussed below:

(1) The Sanskrit Heritage Site: The **Sanskrit Heritage** (**SH**) Site provides a range of services for the computational analysis of Sanskrit, including access to dictionaries, lemmatizers, and a reader. This toolkit incorporates two significant dictionaries, namely the Sanskrit-French Heritage dictionary and the Monier-Williams Sanskrit-English dictionary, along with a morphological lexicon containing inflected forms in XML format under various transliteration schemes. The generative lexicon of SH gave rise to the first hypertext Sanskrit dictionary equipped with grammatical tools such as inflection generation for both the French and English dictionaries, which is a much appreciated feature by the learners of the language. Thus, highlighting the importance of digital lexicons, and their management in computational linguistics. The Sanskrit Heritage Reader also enables machine-assisted analysis of Sanskrit

sentences, encompassing segmentation, morphological tagging, and multiple parsers. While many of these tasks have been discussed in the preceding sections, additional information on the utilization of these dictionaries and tools can be found on the web interface.[23]

(2) Saṃsādhanī: The Department of Sanskrit Studies from the University of Hyderabad has developed a Sanskrit toolkit named Saṃsādhanī.[24] The platform provides a diverse set of tools, including a Sandhi analyzer and generator, morphological analyzer and generator, Aṣṭādhyāyī simulator, transliterator, Nyāyacitradīpikā (an integrated tool for segmenting, parsing, graph rendering, and identifying compound types in Navya-Nyāya expressions), Yamaka and Anuprāsa identifier. Additionally, there is a Sanskrit-to-Hindi accessor cum machine translator named Anusāraka. The platform encompasses the knowledge net of Amarakoṣa (utilizing the Nyāya ontology) and Pāṇinian Dhātuvṛttis as dictionaries, along with the Sanskrit-French Heritage dictionary, Monier Williams Sanskrit-English dictionary, Apte's Sanskrit-Hindi dictionary and Cappller's Sanskrit-German dictionary. Furthermore, the toolkit includes various tagged and untagged corpora for computational analysis, along with sample e-readers on Bhagavadgīta, Sankṣeparāmāyaṇa, Śiśupālavadha, and Raghuvamśa. The toolkit also consists of a **Sanskrit Text Annotation and Research Tool** (**START**) [Kumar et al. 2024] as a mirror site that provides an appropriate user interface for the semi-automatic generation of automated texts, essential for constructing E-readers. The platform further offers access to "Gaveṣikā," the first Sanskrit search engine. This tool enables users to search for Sanskrit words or even prātipadika/dhātu across different corpora. While the implementation details of several tools are discussed in other sections of this article, for a more in-depth understanding, one can also refer to the theses of research students available on the website.[25]

(3) JNU Toolkit: The Research and Development in Computational Linguistics at the School of Sanskrit and Indic Studies, **Jawaharlal Nehru University** (**JNU**), has been actively engaged in diverse areas of language technology for Sanskrit and other Indian languages. Presently, the department is dedicated to the creation of Sanskrit analysis tools for the development of the **Sanskrit-Hindi Translator** (**SaHiT**). Notable tools developed thus far include the Sandhi Splitter, Sandhi Generator, Subanta (nominal form) analyzer, Subanta Generator, Tiṇanta (verbal form) analyzer, Tiṇanta generator, Kṛdanta (derivatives) analyzer, Sanskrit POS tagger, Sanskrit anaphora resolution system, Kāraka analyzer, Gender Recognizer and Sanskrit Homonym Analyzer. These tools have largely emerged from the research theses of students affiliated with the department.[26] The toolkit also integrates access to various lexical resources such as Amarakoṣa, Halāyudhakoṣa, Mahābhārata, Medinīkoṣa, Ayurveda texts, and more. Additionally, the department has been instrumental in the development of tagsets for Indian languages and has introduced the first Sanskrit tagset, which is available on their website.[27]

(4) Sambhāṣā: The Vyākaraṇa department at Karnataka Sanskrit University is dedicated to developing a computational analysis toolkit for Sanskrit.[28] Several projects have been undertaken as part of this initiative, including the Mahākoṣa data navigation tool, Āyussamskṛtam Courseware for Ayurvedic medical students, Tarkasaṅgrahaḥ Ontological knowledge representation, Vāṇmayī-e create text, Prākṛt verbforms generator, Nāmadhāturūpikā

[23] https://sanskrit.inria.fr/
[24] https://sanskrit.uohyd.ac.in/scl/
[25] https://sanskrit.uohyd.ac.in/faculty/amba/#d18
[26] http://sanskrit.jnu.ac.in/rstudents/phd.jsp
[27] http://sanskrit.jnu.ac.in/index.jsp
[28] https://sambhasha.ksu.ac.in/

denominative verbs analyzer and generator, Prācīnanyāyaḥ Ontological knowledge representation, Sanskrit Quiz App, Spell Checker, Bhaṭṭikāvya with Kāraka dependency diagram, Anvayacitraṇam tool, Vaijayantīkośa Knowledge net, Śabdakautukam Practice Noun forms, Sandhi Splitting tool, and **E-Śloka Architect** (**E-SA**). The website also provides access to e-courses for beginners interested in learning Sanskrit.

(5) SHR++: Krishna et al. [2020c] introduce a web-based annotation framework designed to assist professional annotators in the morpho-syntactic annotation of Sanskrit corpora. This interface makes use of the Sanskrit Heritage Reader and extends its functionalities to include annotation capabilities for dependency relations as well. It leverages the SH to enumerate valid word splits and their morphological analyses, thereby aiding annotators in the decision-making process. It also incorporates segmentation suggestions from automated tools, aiming at reducing the cognitive load of annotators. The tool's evaluation experiment demonstrates a reduction in annotation time by about 20.15%, highlighting its effectiveness in aiding human annotators.

(6) Antarlekhaka: Terdalkar and Bhattacharya [2023b] present a tool that enables manual annotation of Sanskrit corpora in eight different categories. The task categories supported by Antarlekhaka include sentence boundary detection, canonical word ordering, free-form text annotation of tokens, token classification, token graph construction, token connection, sentence classification, and sentence graph construction. The tool is designed to be unicode-compatible, language-agnostic, and deployable on the web, allowing for distributed annotation by multiple annotators simultaneously. Its performance was assessed through a comprehensive two-fold evaluation method, encompassing both subjective and objective measures. In the subjective evaluation, 16 annotators rated the tool on a scale of 1 to 5 across various categories, including ease of use, annotation interface, and overall performance. The tool achieved scores of 4.3 for ease of use, 4.4 for the annotation interface, and an overall score of 4.1. Objectively, the tool was benchmarked against other existing tools across 29 categories, securing a total score of 0.79, surpassing the performance of all other tools.

Courses are being offered on how to use these computational platforms,[29] catering to students and others interested in learning Sanskrit. This makes it easier for them to apply these tools for various purposes, enhancing their learning experience. Although these courses aim at explaining the tools of the software platforms, they are also helpful for spotting errors and correcting the software, based on the feedback. Additionally, some exercises included in these courses have been systematically used for corpus tagging through crowd-sourcing, facilitating faster data annotation and evaluation.

12.2 Digital Libraries and Repositories for Training Software

Besides the data resources that can be found as a part of the computational platforms mentioned earlier, several machine readable Sanskrit texts have been made available. The advent of SCL began with the digitization of texts in Sanskrit, and the primary efforts in this direction have been made by:

(1) The GRETIL:[30] GRETIL is a platform that offers machine-readable texts in Indian languages which includes Sanskrit as well. These texts come from different people and organizations. It started as a register of download sites for texts but has now been focusing on documenting and encoding these texts. It makes sure the texts are easy to search across all languages in the collection, without needing extra conversion.

[29] Samsādhanī – Praveśikā https://sanskrit.uohyd.ac.in/IOE102/
[30] https://gretil.sub.uni-goettingen.de/gretil.html

(2) Sanskrit Library:[31] Sanskrit Library is a platform committed to advancing education and research in Sanskrit, offering access to digitized texts and computerized tools. The platform consists of tools such as encoder, morphological analyzer, meter identification tool, and nominal and verbal paradigm generation. The Sanskrit Library also offers courses for those interested in enhancing their learning of Sanskrit and other Indian languages.
(3) SARIT[32] and Ambuda[33]: We also have initiatives such as **Search and retrieval of Indic texts** (**SARIT**) and Ambuda. SARIT hosts electronic versions of Sanskrit texts that are well-documented, dated, and contain embedded notes regarding their revision history. These texts are marked using the **Text Encoding Initiative** (**TEI**), which serves multiple purposes and remains adaptable and valuable in the long run. Ambuda, on the other hand, is a comprehensive library of traditional Sanskrit literature. While it is relatively smaller in scale at present, its goal is to become the largest Sanskrit e-library, offering access to Sanskrit texts of the highest quality from anywhere in the world. It includes features such as word-by-word analysis, integrated dictionary support, and transliteration into various scripts.

We also have multiple other repositories in Sanskrit such as the Wordnet, as well as datasets that aid in training the models (softwares). These include:

(1) **Digital Corpus of Sanskrit** (**DCS**):[34] Another major dataset repository for Sanskrit is the DCS. It consists of Sanskrit texts that have undergone Sandhi-splitting, providing comprehensive morphological and lexical analysis. This is the largest annotated dataset available for Sanskrit. It also has access to an online dictionary.
(2) Sanskrit Wordnet: Inspired by the English wordnet which was developed at Princeton University, attempts were made to develop wordnets for Indian languages as well. A wordnet for Sanskrit is being developed at IIT Bombay.[35] Sanskrit Wordnet relies on the Hindi Wordnet as its primary source. Initially, the creation of synsets in Sanskrit Wordnet involved using random synsets from the Hindi Wordnet. Subsequently, curated lists of significant Sanskrit words were obtained from various sources. Innovative approaches were implemented, including the insertion of concepts or glosses into the Sanskrit Wordnet. To generate synset glosses in Sanskrit, a combination of glosses from dictionaries like Śabdakalpadruma and translations of glosses from Hindi Wordnet synsets were employed. The strategy for synset creation followed the principle of commencing with the most frequently used words in modern Sanskrit and concluding the synset with the least frequent words from Vedic Sanskrit [Kulkarni et al. 2010a]. Creating the Sanskrit Wordnet poses significant challenges, primarily attributed to the extensive volume of lexical knowledge. Therefore the development of Sanskrit Wordnet is an ongoing process. Another WordNet[36] for Sanskrit has also been developed under the guidance of Dr. William Michael Short at the University of Exeter. This WordNet aims at modeling Sanskrit's semantic system as comprehensively and accurately as possible, in a format that is both machine-interpretable and machine-actionable. This makes it suitable for use in various NLP applications, particularly in the field of natural language understanding.
(3) Other Datasets: To align data with their experiments on word segmentation, researchers have generated subsets from the DCS called the SIGHUM dataset. Notable among them are

[31] https://sanskritlibrary.org/index.html
[32] https://sarit.indology.info/sarit-pm/docs/welcome.html
[33] https://ambuda.org/
[34] http://www.sanskrit-linguistics.org/dcs/
[35] https://www.cfilt.iitb.ac.in/wordnet/webswn/english_version.php
[36] https://sanskritwordnet.unipv.it/

DCS (SEGMENTATION) by Hellwig and Nehrdich [2018], DCS (SIGHUM) by Krishna et al. [2017] and DCS (HACKATHON).[37] Additionally, Krishnan et al. [2020] and Krishnan et al. [2024] utilize the DCS to establish a gold-tagged dataset, employing the SHR to create a morphologically tagged and segmented corpus. Apart from these, there are datasets created by MeitY under consortium mode funded by the **Technology Development for Indian languages** (**TDIL**).[38] Furthermore, there's the **Sanskrit Treebank Corpus** (**STBC**), sourced from UoH for dependency parsing, and the Universal Dependencies: Vedic Sanskrit Treebank.[39] A dataset named Sandhikosh [Bhardwaj et al. 2018] has been formulated specifically for evaluating the performance of Sanskrit Sandhi tools. This dataset comprises five subcorpora, encompassing a rule-based corpus, literature corpus, Bhagavad Gīta corpus, UoH corpus, and Aṣṭādhyāyī corpus. We also have initiatives undertaken for data creation by the **Central Institute of Indian Languages** (**CIIL**)[40] and the AI4BHARAT.[41]

13 Forging Ahead: Observations and Future Work

Being a field that was almost non-existent fifteen years ago, SCL has rapidly evolved to encompass a wide range of models, from rule-based systems to advanced pre-trained ones. We have multiple digital resources and repositories available today that can be made use of for several purposes. The growth of the field is so dynamic that it fosters frequent collaborative efforts. Datasets are now openly available, software and models are shared on open-source platforms, and many of these resources can even be cloned for local use. A key factor driving this collaborative spirit is the organization of regular events such as the Sanskrit Computational Linguistics Symposium series,[42] along with dedicated sessions at the World Sanskrit Conference[43] and various other workshops. These events bring together Sanskrit scholars, computer scientists, and linguists, allowing them to engage in discussions that maintain the foundations (language theories, grammatical frameworks, and more) of the field. These foundations are crucial for conceptualizing problem statements and creating the necessary data. Initially, the field lacked sufficient annotated data to even begin with any kind of computational analysis. Over the past 15 years, researchers have dedicated themselves to addressing this gap by conceptualizing problem statements and creating datasets, ranging from digitizing texts to annotation. Each effort resulted in unique annotation schemas, based on their varied methodologies and approaches. Thus, both data creation and implementation needed to progress simultaneously. On the other hand, widely spoken languages had access to sufficient data, which they leveraged to build numerous models. While the methodologies used in NLP today are also applied in SCL, comparing the results of NLP in general to this field would be non trivial at present, given the differing focuses of the two fields. We hope this will be the subject of a more detailed study in the future.

We now summarize our observations on the various tasks discussed in this article:

In Sanskrit, word segmentation, morphological analysis, and compound analysis all go hand-in-hand. Addressing these as joint tasks would be helpful as opposed to resolving them as standalone tasks. Additionally, the true performance of these models can only be evaluated in real-world applications, such as Machine Translation, where all these tasks come together. Therefore, there is

[37] https://sanskritpanini.github.io/
[38] https://tdil.meity.gov.in/AboutUs.aspx
[39] https://universaldependencies.org/treebanks/sa_vedic/index.html
[40] https://corpora.ciil.org/ldcil.htm
[41] https://ai4bharat.iitm.ac.in/
[42] https://iscls.github.io/
[43] https://www.nepalworldsanskrit.org/

a need for more robust, integrated systems that can effectively leverage these sub-tasks, ensuring better performance across varied linguistic challenges.

The focus of SCL has predominantly centered around morphophonemics, morphosyntactics, and syntax. However, there remains substantial uncharted territory in semantics and discourse analysis. There have been primitive efforts in Machine Translation for Sanskrit, with most of them focusing on the Sanskrit-English pair. Although platforms like Google Translate offer multilingual translation, they are sometimes completely erroneous. This could be attributed to the limited availability of training resources, which is the bottleneck of good MT systems such as Google Translate. Therefore, it is crucial to develop large parallel corpora for Sanskrit and other Indian languages, such that the models can be trained on quality data, thereby improving the model's performance.

Limited attention has been devoted to exploring Keyword Extraction, Anaphora Resolution, Information Retrieval and, Question-Answering systems in Sanskrit. Furthermore, other critical pre-processing tasks, essential for major NLP applications, like Word Sense Disambiguation and Semantic Role Labeling, remain largely unexplored for Sanskrit. Apart from the work by Hellwig [2017] dealing with semantic disambiguation of rare nouns, there have been no concrete efforts in handling polysemy or homonymy so far.

Sanskrit benefited early from a rich tradition of literary criticism that developed notions of sentiment analysis, implicature, suggestion, figures of speech and sound and so forth. Among these, the studies so far only deal with Chandas (prosody) Analysis and Alaṅkāra (figures of speech and sound) Analysis. And even for these tasks, approaches rely on rule-based methods. An intriguing avenue for exploration is the potential use of Machine Learning in these areas. Additionally, models could be developed to identify style, as well as regional, temporal, or sectarian variations in writing, which would be valuable for dating and attributing works. This would be especially beneficial for Sanskrit literature, which faces challenges in these areas. Moreover, addressing specific figures of speech such as "śleṣā" and advanced poetic devices like citrakāvya and vilomakāvya that are both unique to Sanskrit, could provide promising directions for future research.

Tasks such as Named Entity Recognition, Sentiment Analysis, Humor Analysis, Summarization, Paraphrasing and, Dialogue Systems are areas that are yet to be explored. Despite some sentiment and semantic analyses applied to translations of Sanskrit texts, as demonstrated by Chandra and Kulkarni [2022] and Shukla et al. [2023], there exists a notable gap in the direct analysis of Sanskrit texts themselves due to the lack of labelled data. This is yet another area for exploration in the future. Furthermore, restoring accent of texts is another potential application which has not been entertained so far. This could be of great help specifically in compound analysis.

In addition to these challenges, the existing pool of digitized texts and datasets in Sanskrit is significantly limited for machine learning purposes. Therefore, a pivotal direction for the future in Sanskrit involves the creation of more diverse datasets tailored for various NLP tasks. The **Indic Knowledge System** (**IKS**) holds immense conceptual potential for advancing the computational analysis of Sanskrit and other Indian languages. Along with Aṣṭādhyāyī and theories of verbal cognition (Śābdabodha), delving into diverse theories encompassing word and meaning, poetry and aesthetics, and philosophy can significantly strengthen the depth of linguistic analysis applied to Sanskrit.

Our survey, to its best possible extent, has covered most of the computational tasks pertaining to Sanskrit and has given a brief overview of SCL along with the supporting digital framework. However, one significant challenge encountered during the study was the difficulty in quantifying and comparing the results of various models across different sections. This was primarily due to the fact that each study utilized different datasets for training and evaluation, as well as varied metrics to assess model performance. The lack of uniformity in datasets (annotation, size, quality) and evaluation metrics prevented us from making direct comparisons. Additionally, the diversity

of models, ranging from rule-based systems to pre-trained models, further complicated any meaningful comparison of performance. Drawing conclusions solely based on the reported performance seemed unjustified, and we therefore refrained from doing so. This issue presents an opportunity for future work, where a more systematic comparison could be conducted in a subsequent survey article, with standardized datasets and evaluation metrics to facilitate a clearer comparison across models.

Acknowledgement

I would like to thank Prof. Amba Kulkarni for the resources and valuable inputs. I extend my thanks to Mr. Sriram Krishnan for his continuous guidance and support.

References

Rahul Aralikatte, Miryam de Lhoneux, Anoop Kunchukuttan, and Anders Søgaard. 2021. Itihasa: A large-scale corpus for Sanskrit to English translation. In *Proceedings of the 8th Workshop on Asian Translation*. Association for Computational Linguistics, 191–197.

Rahul Aralikatte, Neelamadhav Gantayat, Naveen Panwar, Anush Sankaran, and Senthil Kumar Kumarasamy Mani. 2018. Sanskrit Sandhi splitting using seq2 (seq)^ 2. In *Proceedings of the Conference on Empirical Methods in Natural Language Processing*. Association for Computational Linguistics, 4909–4914.

S. R. Arjuna and Amba Kulkarni. 2016. Analysis and graphical representation of navya-nyāya expressions. In *Sanskrit and Computational Linguistics - 16th World Sanskrit Conference*, Bangkok, Thailand. 21–52.

Promila Bahadur, A. K. Jain, and D. S. Chauhan. 2012. Etrans-a complete framework for english to sanskrit machine translation. In *Proceedings of the International Journal of Advanced Computer Science and Applications from International Conference and workshop on Emerging Trends in Technology*. The Science and Information Organization.

Amruta Barbadikar and Amba Kulkarni. 2024. Yamaka identifier and classifier: A computational tool for the analysis of Sanskrit figure of sound. *Annals of Bhandarkar Oriental Research Institute* 102 (2024), 29–45.

Amruta Vilas Barbadikar and Amba Kulkarni. 2024. Anuprāsa identifier and classifier: A computational tool to analyze Sanskrit figure of sound. In *Proceedings of the 7th International Sanskrit Computational Linguistics Symposium*. Association for Computational Linguistics.

Kenneth R. Beesley. 2004. Morphological analysis and generation: A first step in natural language processing. In *1st Steps in Language Documentation for Minority Languages: Computational Linguistic Tools for Morphology, Lexicon and Corpus Compilation, Proceedings of the SALTMIL Workshop at LREC*. 1–8.

Akshar Bharati, Vineet Chaitanya, Rajeev Sangal, and Brendan Gillon. 2002. *Natural Language Processing: A Paninian Perspective*. PHI Learning Pvt. Ltd.

Akshar Bharati and Amba Kulkarni. 2007. Sanskrit and computational linguistics. In *Proceedings of the 1st International Sanskrit Computational Symposium. Department of Sanskrit Studies, University of Hyderabad.*

Akshar Bharati, Amba P. Kulkarni, and V. Sheeba. 2006. Building a wide coverage Sanskrit Morphological Analyser: A practical approach. In *Proceedings of the 1st National Symposium on Modelling and Shallow Parsing of Indian Languages, IIT-Bombay.*

Akshar Bharati and Rajeev Sangal. 1993. Parsing free word order languages in the Paninian framework. In *Proceedings of the 31st Annual Meeting of the Association for Computational Linguistics*. 105–111.

Shubham Bhardwaj, Neelamadhav Gantayat, Nikhil Chaturvedi, Rahul Garg, and Sumeet Agarwal. 2018. Sandhikosh: A benchmark corpus for evaluating sanskrit sandhi tools. In *Proceedings of the 11th International Conference on Language Resources and Evaluation*. European Language Resources Association (ELRA).

Rohitash Chandra and Venkatesh Kulkarni. 2022. Semantic and sentiment analysis of selected Bhagavad Gita translations using BERT-based language framework. *IEEE Access* 10 (2022), 21291–21315.

Subhash Chandra. 2012. Restructuring of paninian morphological rules for computer processing of sanskrit nominal inflections. In *Proceedings of the Workshop on Indian Language and Data: Resources and Evaluation Workshop Programme*. 39.

Rahul Chingamtotattil and Rajamma Gopikakumari. 2022. Neural machine translation for Sanskrit to Malayalam using morphology and evolutionary word sense disambiguation. *Indonesian Journal of Electrical Engineering and Computer Science* 28, 3 (2022), 1709–1719.

Sushant Dave, Arun Kumar Singh, Dr Prathosh AP, and Prof Brejesh Lall. 2021. Neural compound-word (Sandhi) generation and splitting in Sanskrit language. In *Proceedings of the 3rd ACM India Joint International Conference on Data Science and Management of Data (8th ACM IKDD CODS and 26th COMAD'21)*. Association for Computing Machinery, 171–177.

Leander Girrbach and Jingwen Li. 2022. Word segmentation and morphological parsing for Sanskrit. *CoRR*. Retrieved from https://arxiv.org/abs/2201.12833

Madhav Gopal and Girish Nath Jha. 2014. Resolving anaphors in Sanskrit. In *Human Language Technology Challenges for Computer Science and Linguistics: 5th Language and Technology Conference, LTC 2011, Poznań, Poland, November 25–27, 2011, Revised Selected Papers 5*. Springer, 83–92.

Pawan Goyal, Vipul Arora, and Laxmidhar Behera. 2007. Analysis of Sanskrit text: Parsing and semantic relations. In *Proceedings of the International Sanskrit Computational Linguistics Symposium*. Springer, 200–218.

Pawan Goyal and Gérard Huet. 2013. Completeness analysis of a Sanskrit reader. In *Proceedings of the 5th International Symposium on Sanskrit Computational Linguistics. DK Printworld (P) Ltd.* 130–171.

Pawan Goyal, Gérard Huet, Amba Kulkarni, Peter Scharf, and Ralph Bunker. 2012. A distributed platform for Sanskrit processing. In *Proceedings of the COLING 2012*. Association for Computational Linguistics, 1011–1028.

Pawan Goyal, Amba Kulkarni, and Laxmidhar Behera. 2009. Computer simulation of aṣṭādhyā yī: Some insights. In *Sanskrit Computational Linguistics: 1st and 2nd International Symposia Rocquencourt, France, October 29–31, 2007 Providence, RI, USA, May 15–17, 2008 Revised Selected and Invited Papers*. Springer, 139–161.

Pawan Goyal and R. Mahesh K. Sinha. 2007. A study towards design of an English to Sanskrit machine translation system. In *Proceedings of the International Sanskrit Computational Linguistics Symposium*. Springer, 287–305.

Ashim Gupta, Amrith Krishna, Pawan Goyal, and Oliver Hellwig. 2020. Evaluating neural morphological taggers for sanskrit. In *Proceedings of the 17th SIGMORPHON Workshop on Computational Research in Phonetics, Phonology, and Morphology*. Association for Computational Linguistics.

Oliver Hellwig. 2007. Sanskrittagger: A stochastic lexical and pos tagger for sanskrit. In *Proceedings of the International Sanskrit Computational Linguistics Symposium*. Springer, 266–277.

Oliver Hellwig. 2015. Using recurrent neural networks for joint compound splitting and Sandhi resolution in Sanskrit. In *Proceedings of the 4th Biennial Workshop on Less-resourced Languages*.

Oliver Hellwig. 2017. Coarse semantic classification of rare nouns using cross-lingual data and recurrent neural networks. In *Proceedings of the 12th International Conference on Computational Semantics–Long Papers*.

Oliver Hellwig and Sebastian Nehrdich. 2018. Sanskrit word segmentation using character-level recurrent and convolutional neural networks. In *Proceedings of the 2018 Conference on Empirical Methods in Natural Language Processing*. Association for Computational Linguistics, 2754–2763.

Oliver Hellwig, Sebastian Nehrdich, and Sven Sellmer. 2023. Data-driven dependency parsing of Vedic Sanskrit. *Language Resources and Evaluation* 57, 3 (2023), 1173–1206.

Oliver Hellwig, Salvatore Scarlata, Elia Ackermann, and Paul Widmer. 2020. The treebank of vedic Sanskrit. In *Proceedings of the 12th Language Resources and Evaluation Conference*. 5137–5146.

Md Enamul Hoque. 2021. The universal grammar theory: Noam Chomsky's contribution to second language (SL) education. *The Journal of EFL Education and Research* 6, 2 (2021), 57–62.

Gérard Huet. 2003a. Lexicon-directed segmentation and tagging of Sanskrit. In *Proceedings of the XIIth World Sanskrit Conference, Helsinki, Finland, Aug*. 307–325.

Gérard Huet. 2003b. Towards computational processing of Sanskrit. In *Proceedings of the International Conference on Natural Language Processing*. 40–48.

Gérard Huet. 2004. Design of a lexical database for Sanskrit. In *Proceedings of the Workshop on Enhancing and Using Electronic Dictionaries*. Association for Computational Linguistics.

Gérard Huet. 2005. A functional toolkit for morphological and phonological processing, application to a Sanskrit tagger. *Journal of Functional Programming* 15, 4 (2005), 573–614.

Gérard Huet. 2006. Shallow syntax analysis in Sanskrit guided by semantic nets constraints. In *Proceedings of the 2006 International Workshop on Research Issues in Digital Libraries*. Association for Computing Machinery, 1–10.

Gérard Huet. 2009. Sanskrit Segmentation. In *Proceedings of the South Asian Languages Analysis Roundtable XXVIII* (Denton, Ohio).

Gérard Huet. 2024. Hoisting the colors of Sanskrit. In *Proceedings of the 7th International Sanskrit Computational Linguistics Symposium*. Association for Computational Linguistics.

Girish N. Jha. 2004. Generating nominal inflectional morphology in Sanskrit. *SIMPLE* 4 (2004), 20–23.

Girish Nath Jha, Muktanand Agrawal, Subash, Sudhir K. Mishra, Diwakar Mani, Diwakar Mishra, Manji Bhadra, and Surjit K. Singh. 2009. Inflectional morphology analyzer for Sanskrit. In *Sanskrit Computational Linguistics: 1st and 2nd International Symposia Rocquencourt, France, October 29–31, 2007 Providence, RI, USA, May 15–17, 2008 Revised Selected and Invited Papers*. Springer, 219–238.

Daniel Jurafsky and James H. Martin. 2000. *Speech and Language Processing*. Prentice-Hall.

Mohammed Safi Ur Rahman Khan, Priyam Mehta, Ananth Sankar, Umashankar Kumaravelan, Sumanth Doddapaneni, Suriyaprasaad B, Varun G, Sparsh Jain, Anoop Kunchukuttan, Pratyush Kumar, et al.. 2024. IndicLLMSuite: A blueprint for creating pre-training and fine-tuning datasets for Indian languages. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*. Association for Computational Linguistics.

Nimrita Koul and Sunilkumar S. Manvi. 2021. A proposed model for neural machine translation of Sanskrit into English. *International Journal of Information Technology* 13, 1 (2021), 375–381.

Amrith Krishna and Pawan Goyal. 2015. Towards automating the generation of derivative nouns in Sanskrit by simulating Panini. In *Graph-Based Methods for Natural Language Processing: Proceesings of the Workshop*.

Amrith Krishna, Ashim Gupta, Deepak Garasangi, Jivnesh Sandhan, Pavankumar Satuluri, and Pawan Goyal. 2023. Neural approaches for data driven dependency parsing in Sanskrit. In *Proceedings of the Computational Sanskrit and Digital Humanities: Selected Papers Presented at the 18th World Sanskrit Conference*. Association for Computational Linguistics. 1–20.

Amrith Krishna, Bishal Santra, Sasi Prasanth Bandaru, Gaurav Sahu, Vishnu Dutt Sharma, Pavankumar Satuluri, and Pawan Goyal. 2018. Free as in free word order: An energy based model for word segmentation and morphological tagging in Sanskrit. In *Proceedings of the 2018 Conference on Empirical Methods in Natural Language Processing*. Association for Computational Linguistics.

Amrith Krishna, Bishal Santra, Ashim Gupta, Pavankumar Satuluri, and Pawan Goyal. 2020b. A graph-based framework for structured prediction tasks in Sanskrit. *Computational Linguistics* 46, 4 (2020).

Amrith Krishna, Bishal Santra, Pavankumar Satuluri, Sasi Prasanth Bandaru, Bhumi Faldu, Yajuvendra Singh, and Pawan Goyal. 2016a. Word segmentation in sanskrit using path constrained random walks. In *Proceedings of COLING 2016, the 26th International Conference on Computational Linguistics: Technical Papers*. The COLING 2016 Organizing Committee, 494–504.

Amrith Krishna, Pavankumar Satuluri, and Pawan Goyal. 2017. A dataset for sanskrit word segmentation. In *Proceedings of the Joint SIGHUM Workshop on Computational Linguistics for Cultural Heritage, Social Sciences, Humanities and Literature*. 105–114.

Amrith Krishna, Pavankumar Satuluri, Shubham Sharma, Apurv Kumar, and Pawan Goyal. 2016b. Compound type identification in sanskrit: What roles do the corpus and grammar play?. In *Proceedings of the 6th Workshop on South and Southeast Asian Natural Language Processing*. The COLING 2016 Organizing Committee, 1–10.

Amrith Krishna, Vishnu Sharma, Bishal Santra, Aishik Chakraborty, Pavankumar Satuluri, and Pawan Goyal. 2019. Poetry to prose conversion in Sanskrit as a linearisation task: A case for low-resource languages. In *Proceedings of the 57th Annual Meeting of the Association for Computational Linguistics*. Association for Computational Linguistics.

Amrith Krishna, Shiv Vidhyut, Dilpreet Chawla, Sruti Sambhavi, and Pawan Goyal. 2020c. SHR++: An interface for morpho-syntactic annotation of Sanskrit corpora. In *Proceedings of the 12th Language Resources and Evaluation Conference*. 7069–7076.

Sriram Krishnan and Amba Kulkarni. 2022. Sanskrit segmentation revisited. In *Findings of the Association for Computational Linguistics: EMNLP*. Association for Computational Linguistics.

Sriram Krishnan, Amba Kulkarni, and Gérard Huet. 2020. Validation and normalization of DCS corpus using Sanskrit Heritage tools to build a tagged Gold Corpus. In *Proceedings of the Computational Sanskrit and Digital Humanities: Selected Papers Presented at the 18th World Sanskrit Conference*.

Sriram Krishnan, Amba Kulkarni, and Gérard Huet. 2024. Normalized dataset for Sanskrit word segmentation and morphological parsing. *Language Resources and Evaluation* (2024), 1–52.

Amba Kulkarni. 2013. A deterministic dependency parser with dynamic programming for Sanskrit. In *Proceedings of the Second International Conference on Dependency Linguistics*. Charles University in Prague, 157–166.

Amba Kulkarni. 2016. *Application of Modern Technology to South Asian Languages*. Walter de Gruyter GmbH & Co KG.

Amba Kulkarni, Sheetal Pokar, and Devanand Shukl. 2010b. Designing a constraint based parser for Sanskrit. In *Sanskrit Computational Linguistics: 4th International Symposium, New Delhi, India, December 10–12, 2010. Proceedings*. Springer, 70–90.

Amba Kulkarni, Sanal Vikram, and K. Sriram. 2019. Dependency parser for Sanskrit verses. In *Proceedings of the 6th International Sanskrit Computational Linguistics Symposium*. 14–27.

Malhar Kulkarni, Chaitali Dangarikar, Irawati Kulkarni, Abhishek Nanda, and Pushpak Bhattacharyya. 2010a. Introducing sanskrit wordnet. In *Proceedings of the 5th Global Wordnet Conference, Narosa, Mumbai*. 287–294.

Anil Kumar. 2012. *An Automatic Sanskrit Compound Processing*. Ph. D. Dissertation. University of Hyderabad.

Anil Kumar and Amba Kulkarni. 2010. Sanskrit compound processor. In *Sanskrit Computational Linguistics: 4th International Symposium, New Delhi, India, December 10–12, 2010. Proceedings*. Springer, 57–69.

Anil Kumar, Amba Kulkarni, and Nakka Shailaj. 2024. START: Sanskrit teaching; Annotation; and research tool – bridging tradition and technology in scholarly exploration. In *Proceedings of the 7th International Sanskrit Computational Linguistics Symposium*. Association for Computational Linguistics.

Anil Kumar, V. Sheebasudheer, and Amba Kulkarni. 2009. Sanskrit compound paraphrase generator. *Proceedings of 7th International Conference on Natural Language Processing (ICON'09)*.

Rashi Kumar, Piyush Jha, and Vineet Sahula. 2019. An augmented translation technique for low resource language pair: Sanskrit to hindi translation. In *Proceedings of the 2019 2nd International Conference on Algorithms, Computing and Artificial Intelligence*. Association for Computing Machinery, 377–383.

Sachin Kumar. 2007. Sandhi splitter and analyzer for Sanskrit (with reference to ac Sandhi). *Mphil Degree at SCSS, JNU (Submitted, 2007)* (2007).

Deepika Kumawat and Vinesh Jain. 2015. POS tagging approaches: A comparison. *International Journal of Computer Applications* 118, 6 (2015), 32–38.

D. Mane, P. Devale, and S. Suryawans. 2010. A design towards English to Sanskrit machine translation and synthesizer system using rule base approach. *International Journal of Multidisciplinary Research and Advancement in Engineering* 2, 2 (2010), 405–414.

Keshav S. Melnad, Pawan Goyal, and Peter Scharf. 2013. Meter identification of Sanskrit verse. *Seminar on Sanskrit Syntax and Discourse Structures*. The Sanskrit Library, USA (2013).

Anand Mishra. 2007. Simulating the pāṇinian system of sanskrit grammar. In *Proceedings of the International Sanskrit Computational Linguistics Symposium*. Springer, 127–138.

Anand Mishra. 2009. Modelling the grammatical circle of the pāṇinian system of Sanskrit grammar. In *Sanskrit Computational Linguistics: Third International Symposium, Hyderabad, India, January 15–17, 2009. Proceedings*. Springer, 40–55.

Anand Mishra. 2010. Modelling Aṣṭādhyāyī: An approach based on the methodology of ancillary disciplines (Vedāṅga). In *Proceedings of the International Sanskrit Computational Linguistics Symposium*. Springer, 239–258.

Vimal Mishra and R. B. Mishra. 2010. ANN and rule based model for English to Sanskrit machine translation. *INFOCOMP Journal of Computer Science* 9, 1 (2010), 80–89.

Ruslan Mitkov. 1999. *Anaphora Resolution: The State of the Art*. School of Languages and European Studies, University of Wolverhampton ....

Vipul Mittal. 2010. Automatic Sanskrit segmentizer using finite state transducers. In *Proceedings of the ACL 2010 Student Research Workshop*. Association for Computational Linguistics, 85–90.

Diego Mollá and José Luis Vicedo. 2006. Special section on restricted-domain question answering. *Computational Linguistics* 33, 1 (2006), 41–61.

Abhiram Natarajan and Eugene Charniak. 2011. S3-Statistical Sandhi splitting-statistical Sandhi splitting. In *Proceedings of the 5th International Joint Conference on Natural Language Processing*. Asian Federation of Natural Language Processing, 301–308.

Sebastian Nehrdich, Oliver Hellwig, and Kurt Keutzer. 2024. One model is all you need: ByT5-Sanskrit, a unified model for Sanskrit NLP tasks. In *Findings of the Association for Computational Linguistics: EMNLP 2024*. Association for Computational Linguistics.

Rajneesh Kumar Pandey and Girish Nath Jha. 2016. Error analysis of sahit-a statistical sanskrit-hindi translator. *Procedia Computer Science* 96, C (2016), 495–501.

Dhaval Patel and Shivakumari Katuri. 2015. Prakriyāpradarśinī-an open source subanta generator. In *Sanskrit and Computational Linguistics-16th World Sanskrit Conference, Bangkok, Thailand*.

Ganesh R. Pathak and Sachin P. Godse. 2010. English to Sanskrit machine translation using transfer based approach. In *AIP Conference Proceedings*. American Institute of Physics, 122–126.

Anagha Pradeep, Radhika Mamidi, and Pavankumar Satuluri. 2024. Context and WSD: Analysing Google Translate's Sanskrit to English output of bhagavadgita verses for word meaning. In *Proceedings of the 7th International Sanskrit Computational Linguistics Symposium*. Association for Computational Linguistics.

Pravin Pralayankar and Sobha Lalitha Devi. 2010. Anaphora resolution algorithm for Sanskrit. In *Sanskrit Computational Linguistics: 4th International Symposium, New Delhi, India, December 10–12, 2010. Proceedings*. Springer, 209–217.

Arun K. Prasad. 2024. A fast prakriyā generator. In *Proceedings of the 7th International Sanskrit Computational Linguistics Symposium*. Association for Computational Linguistics.

Abhinandan S. Prasad and Shrisha Rao. 2010. Citation matching in Sanskrit corpora using local alignment. In *Sanskrit Computational Linguistics: 4th International Symposium, New Delhi, India, December 10–12, 2010. Proceedings*. Springer, 124–136.

S. Prasanna. 2022. Spellchecker for Sanskrit: The road less taken. In *Proceedings of the 19th International Conference on Natural Language Processing*. 290–299.

B. Premjith, Chandni Chandran, Shriganesh Bhat, Soman Kp, and P. Prabaharan. 2019. A machine learning approach for identifying compound words from a Sanskrit text. In *Proceedings of the 6th International Sanskrit Computational Linguistics Symposium*. Association for Computational Linguistics, 45–51.

B. Premjith, K. P. Soman, and Prabaharan Poornachandran. 2018. A deep learning based Part-of-Speech (POS) tagger for Sanskrit language by embedding character level features. In *Proceedings of the FIRE*. 56–60.

Ravneet Punia, Aditya Sharma, Sarthak Pruthi, and Minni Jain. 2020. Improving neural machine translation for Sanskrit-English. In *Proceedings of the 17th International Conference on Natural Language Processing*. NLP Association of India (NLPAI), 234–238.

V. Raghavan. 1942. *Studies on Some Concepts of the Alaṅkāra Śāstra*. The Adyar Library.

Shreevatsa Rajagopalan. 2020. A user-friendly tool for metrical analysis of Sanskrit verse. *Computational Sanskrit and Digital Humanities* (2020), 113.

N. Rama and Meenakshi Lakshmanan. 2010. A computational algorithm for metrical classification of verse. *International Journal of Computer Science Issues* 7 (2010), 46–53.

Jaideepsinh K. Raulji, Jatinderkumar R. Saini, Kaushika Pal, and Ketan Kotecha. 2022. A novel framework for Sanskrit-Gujarati symbolic machine translation system. *International Journal of Advanced Computer Science and Applications* 13, 4 (2022).

Vikas Reddy, Amrith Krishna, Vishnu Dutt Sharma, Prateek Gupta, Pawan Goyal, et al. 2018. Building a word segmenter for Sanskrit overnight. In *Proceedings of the 11th International Conference on Language Resources and Evaluation*. European Language Resources Association (ELRA'18), 1724–1734.

Jivnesh Sandhan, Ashish Gupta, Hrishikesh Terdalkar, Tushar Sandhan, Suvendu Samanta, Laxmidhar Behera, and Pawan Goyal. 2022a. A novel multi-task learning approach for context-sensitive compound type identification in Sanskrit. In *Proceedings of the 29th International Conference on Computational Linguistics*. International Committee on Computational Linguistics.

Jivnesh Sandhan, Amrith Krishna, Pawan Goyal, and Laxmidhar Behera. 2019. Revisiting the role of feature engineering for compound type identification in Sanskrit. In *Proceedings of the 6th International Sanskrit Computational Linguistics Symposium*. Association for Computational Linguistics, 28–44.

Jivnesh Sandhan, Amrith Krishna, Ashim Gupta, Laxmidhar Behera, and Pawan Goyal. 2021. A little pretraining goes a long way: A case study on dependency parsing task for low-resource morphologically rich languages. In *Proceedings of the 16th Conference of the European Chapter of the Association for Computational Linguistics: Student Research Workshop*. Association for Computational Linguistics.

Jivnesh Sandhan, Yaswanth Narsupalli, Sreevatsa Muppirala, Sriram Krishnan, Pavankumar Satuluri, Amba Kulkarni, and Pawan Goyal. 2023. DepNeCTI: Dependency-based nested compound type identification for Sanskrit. In *Findings of the Association for Computational Linguistics*. Association for Computational Linguistics.

Jivnesh Sandhan, Rathin Singha, Narein Rao, Suvendu Samanta, Laxmidhar Behera, and Pawan Goyal. 2022b. TransLIST: A transformer-based linguistically informed Sanskrit tokenizer. In *Findings of the Association for Computational Linguistics*. Association for Computational Linguistics.

Pavankumar Satuluri and Amba Kulkarni. 2013. Generation of Sanskrit compounds. *Proceedings of the ICON* (2013), 77–86.

Peter Scharf. 2015. An XML formalization of the Aṣṭādhyāyī. In *Proceedings of the 16th World Sanskrit Conference*.

Nandini Sethi, Amita Dev, and Poonam Bansal. 2023. A novel neural machine translation approach for low-resource Sanskrit-Hindi language pair. *ACM Transactions on Asian and Low-Resource Language Information Processing* (2023).

Akshat Shukla, Chaarvi Bansal, Sushrut Badhe, Mukul Ranjan, and Rohitash Chandra. 2023. An evaluation of Google translate for Sanskrit to English translation via sentiment and semantic analysis. *Natural Language Processing Journal* 4 (2023).

Muskaan Singh, Ravinder Kumar, and Inderveer Chana. 2019. Improving neural machine translation using rule-based machine translation. In *Proceedings of the 2019 7th International Conference on Smart Computing and Communications*. 1–5.

Muskaan Singh, Ravinder Kumar, and Inderveer Chana. 2020. Corpus based machine translation system with deep neural network for Sanskrit to Hindi translation. *Procedia Computer Science* 167 (2020), 2534–2544.

Sitender and Seema Bawa. 2021. A Sanskrit-to-English machine translation using hybridization of direct and rule-based approach. *Neural Computing and Applications* 33, 7 (2021), 2819–2838.

Marco Antonio Calijorne Soares and Fernando Silva Parreiras. 2020. A literature review on question answering techniques, paradigms and systems. *Journal of King Saud University-Computer and Information Sciences* 32, 6 (2020), 635–646.

H. S. Sreedeepa and Sumam Mary Idicula. 2017. Interlingua based Sanskrit-English machine translation. In *Proceedings of the 2017 International Conference on Circuit, Power and Computing Technologies*. IEEE, 1–5.

S. Sridhar and V. Srinivasa. 2008. Computational structure of Aṣṭādhyāyī and conflict resolution techniques. *Proceedings of the 3rd International Symposium on Sanskrit Computational Linguistics*. Springer-Verlag, 56–65.

Prakhar Srivastava, Kushal Chauhan, Deepanshu Aggarwal, Anupam Shukla, Joydip Dhar, and Vrashabh Prasad Jain. 2018. Deep learning based unsupervised POS tagging for Sanskrit. In *Proceedings of the 2018 International Conference on Algorithms, Computing and Artificial Intelligence*. 1–6.

Sridhar Subbanna and Shrinivasa Varakhedi. 2010. Asiddhatva principle in computational model of aṣṭādhyāyī. In *Proceedings of the International Sanskrit Computational Linguistics Symposium*. Springer, 231–238.

Namrata Tapaswi and Suresh Jain. 2012. Treebank based deep grammar acquisition and part-of-speech tagging for Sanskrit sentences. In *Proceedings of the 2012 CSI 6th International Conference on Software Engineering*. IEEE, 1–4.

Namrata Tapaswi, Suresh Jain, and Vaishali Chourey. 2012. Morphological-based spellchecker for Sanskrit sentences. *International Journal of Scientific and Technology Research* 1, 3 (2012), 1–4.

Hrishikesh Terdalkar and Arnab Bhattacharya. 2022. Chandojnanam: A Sanskrit meter identification and utilization system. In *Proceedings of the Computational Sanskrit and Digital Humanities: Selected Papers Presented at the 18th World Sanskrit Conference*. Association for Computational Linguistics.

Hrishikesh Terdalkar and Arnab Bhattacharya. 2023b. Antarlekhaka: A comprehensive tool for multi-task natural language annotation. In *Proceedings of the 3rd Workshop for Natural Language Processing Open Source Software*.

Hrishikesh Terdalkar and Arnab Bhattacharya. 2023a. Framework for question-answering in Sanskrit through automated construction of knowledge graphs. In *Proceedings of the 6th International Sanskrit Computational Linguistics Symposium*. Association for Computational Linguistics.

Neill Tyler. 2023. Skrutable: Another step toward effective Sanskrit meter identification. In *Proceedings of the Computational Sanskrit and Digital Humanities: Selected Papers Presented at the 18th World Sanskrit Conference*. 97–112.

Received 3 April 2024; revised 1 March 2025; accepted 1 April 2025